Nguyễn Đình Tùng Lâm — Đàm Thuận Phong — Nghiêm Tuấn Dũng

DJMNATBU

HSGS Bac Ninh

ICPC 2026

Mục lục

1 Hình học **2**
1.1 Điểm, vector, tích vô hướng và tích có hướng . . . 2
1.2 Đường thẳng, đoạn thẳng và giao điểm . . . 2
1.3 Khoảng cách . . . 2
1.4 Diện tích đa giác đơn . . . 2
1.5 Bao lồi: Graham scan . . . 2
1.6 Đường tròn . . . 3
1.7 Sai số số thực và các trường hợp suy biến . . . 4

2 Xác suất và kỳ vọng **4**
2.1 Xác suất có điều kiện và tính độc lập . . . 4
2.2 Biến chỉ báo và tính tuyến tính của kỳ vọng . . . 4
2.3 Quy hoạch động xác suất . . . 4
2.4 Quy hoạch động và phương trình kỳ vọng . . . 4

3 Lý thuyết trò chơi **5**
3.1 Trạng thái thắng, thua và hòa . . . 5
3.2 Nim . . . 5
3.3 Số Grundy và mex . . . 5
3.4 Trò chơi trên đồ thị có chu trình . . . 5

4 Tổ hợp **6**
4.1 Tổ hợp và đếm cách phân phối . . . 6
4.2 Số Catalan . . . 6
4.3 Bao hàm–loại trừ . . . 6
4.4 Định lý Burnside . . . 6
4.5 Nội suy Lagrange . . . 7

5 Số học **7**
5.1 Euclid mở rộng và nghịch đảo modulo . . . 7
5.2 Hệ phương trình đồng dư và CRT . . . 8
5.3 Số nguyên lớn trong C++ và Python . . . 8

6 Đại số tuyến tính **9**
6.1 Khử Gauss và số nghiệm . . . 9
6.2 Khử Gauss trên GF(2) . . . 9
6.3 Lũy thừa ma trận . . . 10

7 Luồng và lát cắt **11**
7.1 Mô hình mạng luồng . . . 11
7.2 Luồng cực đại và lát cắt cực tiểu . . . 11
7.3 Thuật toán Dinic . . . 11
7.4 Các mẫu dựng mạng từ đề bài . . . 12
7.5 Luồng có cận dưới . . . 12
7.6 Luồng chi phí cực tiểu . . . 12

8 Cặp ghép **13**
8.1 Cặp ghép hai phía . . . 13
8.2 Đường tăng và thuật toán Hopcroft–Karp . . . 13
8.3 Định lý König . . . 14
8.4 Bài toán gán có trọng số . . . 14

9 Logic trên đồ thị **14**
9.1 Mệnh đề và đồ thị kéo theo . . . 14
9.2 Bài toán 2-SAT . . . 15
9.3 Dựng và đọc một nghiệm . . . 15

10 Xử lý chuỗi **15**
10.1 Z-function . . . 15
10.2 Thuật toán Manacher . . . 16
10.3 Trie . . . 16
10.4 Thuật toán Aho–Corasick . . . 16
10.5 Suffix array . . . 17

11 Tối ưu quy hoạch động **18**
11.1 Convex Hull Trick . . . 18
11.2 Li Chao tree . . . 18
11.3 Các tối ưu khác . . . 19

12 Tra cứu nhanh **20**
12.1 Dấu hiệu nhận dạng dạng bài . . . 20
12.2 Gợi ý tư duy ad hoc và constructive . . . 20
12.3 Công thức thường quên . . . 20
12.4 Checklist lỗi và trường hợp biên . . . 20
12.5 Cú pháp Python cho số nguyên lớn . . . 21
12.6 trick lỏ . . . 21

13 Python cho lập trình thi đấu **22**
13.1 Nhập và xuất . . . 22
13.2 Vòng lặp, dãy và chuỗi . . . 22
13.3 Sắp xếp và tìm vị trí . . . 22
13.4 Từ điển, tập hợp, hàng đợi và heap . . . 22
13.5 Các lỗi cú pháp dễ gặp . . . 22

Code C++17; mọi mẫu giả định đã có `#include <bits/stdc++.h>`, `#include <cassert>` *và* `using namespace std;`. *Trong phần lý thuyết, chỉ số thường bắt đầu từ* 1. *Các mẫu code dùng chỉ số từ* 0 *sẽ được ghi rõ tại chỗ.*

1 Hình học

1.1 Điểm, vector, tích vô hướng và tích có hướng

Điểm $P = (x, y)$ cũng biểu diễn vector từ gốc tọa độ tới P. Với A, B là hai điểm, $\overrightarrow{AB} = B - A$. Đường thẳng qua hai điểm *phân biệt* A, B có dạng tham số $P(t) = A + t(B - A)$, $t \in \mathbb{R}$; đoạn AB tương ứng $0 \le t \le 1$. Đường tròn tâm O, bán kính $r \ge 0$ gồm các điểm thỏa $(x - O_x)^2 + (y - O_y)^2 = r^2$.

Với $u = (u_x, u_y)$ và $v = (v_x, v_y)$:

$$u \cdot v = u_x v_x + u_y v_y, \qquad u \times v = u_x v_y - u_y v_x.$$

$u \cdot v$ cho góc, hình chiếu và khoảng cách; $u \times v$ cho hướng quay và hai lần diện tích tam giác. Đặt $\text{orient}(A, B, C) = (B - A) \times (C - A)$: giá trị dương là rẽ trái (ngược chiều kim đồng hồ), âm là rẽ phải, bằng 0 là thẳng hàng. Với tọa độ nguyên, hãy xét dấu *chính xác* thay vì dùng ε.

1.2 Đường thẳng, đoạn thẳng và giao điểm

Đường thẳng qua $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$ cũng có thể viết $Ax + By = C$, trong đó

$$A = y_2 - y_1, \quad B = x_1 - x_2, \quad C = Ax_1 + By_1.$$

Với $L_1 = P + tu$, $L_2 = Q + sv$, nếu $u \times v \neq 0$ thì giao điểm duy nhất là

$$P + u\,\frac{(Q - P) \times v}{u \times v}.$$

Nếu $u \times v = 0$, hai đường song song hoặc trùng nhau; kiểm tra thêm $(Q - P) \times u = 0$ để nhận biết trùng nhau. Không áp dụng công thức cho đường có vector chỉ phương bằng 0.

Đường thẳng AB cắt đoạn CD tại một điểm nếu C, D ở hai phía khác nhau hoặc có đầu mút trên đường; nếu cả hai cùng nằm trên đường thì giao là cả đoạn (hoặc một điểm nếu $C = D$). Với *hai đoạn*, trước hết xét dấu orient của mỗi cặp đầu mút đối với đường chứa đoạn kia; trường hợp bằng 0 phải kiểm tra đầu mút có nằm trong hộp tọa độ của đoạn hay không. Code bên dưới tính cả trường hợp chạm đầu mút, chồng đoạn và đoạn chỉ có một điểm.

1.3 Khoảng cách

Với $u = B - A \neq 0$, khoảng cách từ Q tới đường thẳng AB là

$$d(Q, AB) = \frac{|u \times (Q - A)|}{\|u\|}.$$

Với *đoạn* AB, lấy $t = ((Q - A) \cdot u)/(u \cdot u)$ rồi chặn t vào $[0, 1]$; điểm gần nhất là $A + tu$. Nếu $A = B$, khoảng cách là $\|Q - A\|$. So sánh khoảng cách bình phương khi không cần chính giá trị căn bậc hai.

1.4 Diện tích đa giác đơn

Cho đa giác *không tự cắt* có n đỉnh $P_0, \ldots, P_{n-1}$ theo thứ tự quanh biên; đặt $P_n = P_0$. Hai cách viết tương đương của *hai lần diện tích có dấu* là

$$S_2 = \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) = \sum_{i=0}^{n-1} (x_i - x_{i+1})(y_i + y_{i+1}).$$

Hai tổng bằng nhau vì các hạng $x_i y_i$ triệt tiêu khi lấy tổng theo chu trình. $S_2 > 0$ khi duyệt ngược chiều kim đồng hồ, $S_2 < 0$ khi duyệt thuận chiều; diện tích là $|S_2|/2$. Với tọa độ nguyên, S_2 nguyên nên diện tích là số nguyên hoặc số nguyên cộng 0.5. Nếu đa giác tự cắt, $|S_2|/2$ nói chung không phải diện tích miền.

1.5 Bao lồi: Graham scan

Bao lồi là tập lồi nhỏ nhất chứa mọi điểm; nếu không suy biến, nó là một đa giác lồi. Graham scan: bỏ điểm trùng, chọn đỉnh thấp nhất (nếu hòa chọn trái nhất), sắp xếp các điểm còn lại theo góc cực quanh đỉnh đó, rồi duy trì một stack; khi ba điểm cuối không rẽ trái thì bỏ điểm giữa. Với các điểm cùng tia, sắp xếp gần trước và loại điểm gần. Độ phức tạp $O(n \log n)$; code trả các đỉnh theo chiều ngược kim đồng hồ, không lặp đỉnh đầu, và bỏ các điểm thẳng hàng nằm giữa trên cạnh. Nếu mọi điểm thẳng hàng, kết quả là hai đầu mút (hoặc một điểm nếu tất cả trùng nhau).

Cài đặt số nguyên (C++17). Các phép nhân và hiệu tọa độ được nâng lên `__int128` trước khi tính; vẫn cần kiểm tra giới hạn dữ liệu để tổng cuối không vượt kiểu này. `polygonArea2` dùng đúng công thức S_2 ở trên.

```
using i128 = __int128_t;

struct IP {
    long long x, y;
    bool operator==(const IP& b) const { return x == b.x && y == b.y; }
    bool operator<(const IP& b) const {
        return x != b.x ? x < b.x : y < b.y;
    }
};

i128 cross(IP a, IP b, IP c) {
    i128 ux = (i128)b.x - a.x, uy = (i128)b.y - a.y;
    i128 vx = (i128)c.x - a.x, vy = (i128)c.y - a.y;
    return ux * vy - uy * vx;
}
i128 dot(IP a, IP b, IP c) {
    i128 ux = (i128)b.x - a.x, uy = (i128)b.y - a.y;
    i128 vx = (i128)c.x - a.x, vy = (i128)c.y - a.y;
    return ux * vx + uy * vy;
}
i128 dist2(IP a, IP b) {
    i128 dx = (i128)a.x - b.x, dy = (i128)a.y - b.y;
```

```
    return dx * dx + dy * dy;
}
int sign(i128 x) { return (x > 0) - (x < 0); }
bool onSegment(IP a, IP b, IP p) {
    return cross(a, b, p) == 0 &&
        min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
        min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
bool segmentsIntersect(IP a, IP b, IP c, IP d) {
    int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
    int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
    if (!x && onSegment(a, b, c)) return true;
    if (!y && onSegment(a, b, d)) return true;
    if (!z && onSegment(c, d, a)) return true;
    if (!w && onSegment(c, d, b)) return true;
    return x * y < 0 && z * w < 0;
}
// Duong thang AB va doan CD: 0 khong giao, 1 diem, 2 ca doan CD.
int lineSegmentRelation(IP a, IP b, IP c, IP d) {
    int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
    if (!x && !y) return c == d ? 1 : 2;
    return (!x || !y || x != y) ? 1 : 0;
}

// Hai lan dien tich co dau; duong neu thu tu dinh nguoc kim dong ho.
i128 polygonArea2(const vector<IP>& p) {
    i128 s = 0;
    for (int i = 0, n = (int)p.size(); i < n; ++i) {
        int j = (i + 1) % n;
        s += ((i128)p[i].x - p[j].x) * ((i128)p[i].y + p[j].y);
    }
    return s;
}
// Graham scan: bao loi nguoc kim dong ho, khong lap dinh dau.
vector<IP> grahamHull(vector<IP> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    if (p.size() <= 1) return p;
    auto base = min_element(p.begin(), p.end(), [](IP a, IP b) {
        return a.y != b.y ? a.y < b.y : a.x < b.x;
    });
    swap(p[0], *base);
    IP o = p[0];
    sort(p.begin() + 1, p.end(), [o](IP a, IP b) {
        i128 t = cross(o, a, b);
        return t ? t > 0 : dist2(o, a) < dist2(o, b);
    });
    vector<IP> h;
    for (IP q : p) {
        while (h.size() >= 2 && cross(h[h.size()-2], h.back(), q) <= 0)
            h.pop_back();
        h.push_back(q);
    }
    return h;
}
```

1.6 Đường tròn

Để cắt đường thẳng $L = P + tu$ với đường tròn (O, r), lấy hình chiếu $H = P + u\,((O-P)\cdot u)/(u\cdot u)$ và $d^2 = \|H - O\|^2$. Nếu $d^2 > r^2$ không có giao; bằng nhau thì tiếp xúc tại H; nhỏ hơn thì hai giao điểm là

$$H \pm u\sqrt{\frac{r^2 - d^2}{u \cdot u}}.$$

Muốn giao với *đoạn*, chỉ giữ các nghiệm có tham số $t \in [0, 1]$.

Cài đặt số thực (C++17). `Line` dùng điểm p và vector hướng v; `Circle` dùng tâm và bán kính. Hàm giao hai đường trả rỗng cho cả song song lẫn trùng nhau, nên cần kiểm tra riêng nếu bài đòi phân biệt hai trường hợp. Vector hướng của đường phải khác 0.

```
using ld = long double;
const ld EPS = 1e-12L;

struct FP {
    ld x, y;
    FP operator+(FP b) const { return {x + b.x, y + b.y}; }
    FP operator-(FP b) const { return {x - b.x, y - b.y}; }
    FP operator*(ld k) const { return {x * k, y * k}; }
    FP operator/(ld k) const { return {x / k, y / k}; }
};
ld dot(FP a, FP b) { return a.x * b.x + a.y * b.y; }
ld cross(FP a, FP b) { return a.x * b.y - a.y * b.x; }
ld norm2(FP a) { return dot(a, a); }
ld length(FP a) { return sqrt(norm2(a)); }

struct Line { FP p, v; }; // p + t*v, v != (0,0)
struct Circle { FP c; ld r; };

// nullopt: hai duong song song hoac trung nhau.
optional<FP> lineIntersection(Line a, Line b) {
    ld d = cross(a.v, b.v);
    if (abs(d) <= EPS) return nullopt;
    ld t = cross(b.p - a.p, b.v) / d;
    return a.p + a.v * t;
}
ld distancePointLine(FP q, Line l) {
    return abs(cross(l.v, q - l.p)) / length(l.v);
}
ld distancePointSegment(FP q, FP a, FP b) {
    FP v = b - a;
    if (norm2(v) <= EPS * EPS) return length(q - a);
    ld t = clamp(dot(q - a, v) / norm2(v), 0.0L, 1.0L);
    return length(q - (a + v * t));
}
// Tra ve 0, 1 hoac 2 giao diem voi duong thang vo han.
vector<FP> lineCircleIntersection(Line l, Circle c) {
    ld t = dot(c.c - l.p, l.v) / norm2(l.v);
    FP foot = l.p + l.v * t;
    ld h2 = c.r * c.r - norm2(foot - c.c);
    if (h2 < -EPS) return {};
    if (abs(h2) <= EPS) return {foot};
```

```
    ld shift = sqrt(max((ld)0, h2 / norm2(l.v)));
    return {foot - l.v * shift, foot + l.v * shift};
}
```

2 Xác suất và kỳ vọng

2.1 Xác suất có điều kiện và tính độc lập

Khi các kết quả trong không gian mẫu Ω đồng khả năng, $\Pr(A) = |A|/|\Omega|$; chỉ dùng phép đếm này khi giả thiết *đồng khả năng* thực sự đúng. Chẳng hạn, chọn đều k phần tử khác nhau từ N phần tử, trong đó M phần tử được đánh dấu:

$$\Pr(\text{tất cả đều được đánh dấu}) = \frac{\binom{M}{k}}{\binom{N}{k}}.$$

Công thức trên chỉ áp dụng khi $\binom{N}{k} > 0$.

Với $\Pr(B) > 0$,

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}, \qquad \Pr(A \cap B) = \Pr(A)\Pr(B \mid A) \quad (\Pr(A) > 0).$$

Nếu $B_1, \ldots, B_m$ là một phân hoạch của Ω với $\Pr(B_i) > 0$, công thức xác suất toàn phần và Bayes là

$$\Pr(A) = \sum_i \Pr(A \mid B_i)\Pr(B_i), \quad \Pr(B_j \mid A) = \frac{\Pr(A \mid B_j)\Pr(B_j)}{\Pr(A)}$$

(về sau yêu cầu $\Pr(A) > 0$). Hai biến cố A, B độc lập khi $\Pr(A \cap B) = \Pr(A)\Pr(B)$. *Xung khắc* nghĩa là $A \cap B = \varnothing$; hai khái niệm này khác nhau. Khi lấy mẫu không hoàn lại, các lần chọn thường không độc lập.

2.2 Biến chỉ báo và tính tuyến tính của kỳ vọng

Với biến ngẫu nhiên rời rạc X có kỳ vọng hữu hạn, $\mathbb{E}[X] = \sum_x x \Pr(X = x)$. Đặt $I_A = 1$ nếu A xảy ra, bằng 0 nếu không; khi đó $\mathbb{E}[I_A] = \Pr(A)$. Nếu X đếm số đối tượng thỏa tính chất nào đó, viết $X = \sum_i I_i$ để được

$$\mathbb{E}[X] = \sum_i \Pr(I_i = 1).$$

Tính tuyến tính $\mathbb{E}[\sum_i X_i] = \sum_i \mathbb{E}[X_i]$ *không đòi hỏi các biến độc lập.* Ví dụ: chọn đều k trong N phần tử, M phần tử được đánh dấu. Số phần tử đánh dấu được chọn có kỳ vọng $M \cdot k/N$.

Nếu X nhận giá trị nguyên không âm và có kỳ vọng hữu hạn,

$$\mathbb{E}[X] = \sum_{t \geq 1} \Pr(X \geq t).$$

Công thức này hữu ích khi xác suất của biến cố đạt ít nhất t dễ tính hơn phân phối của X.

1.7 Sai số số thực và các trường hợp suy biến

Dùng phép tính nguyên và `__int128` cho hướng quay, giao đoạn, bao lồi và diện tích khi tọa độ nguyên; kiểm tra giới hạn để tránh tràn ngay cả với kiểu này. Khi cần giao điểm hoặc căn bậc hai, dùng `long double` và chọn ε theo thang tọa độ của bài. Không dùng so sánh có ε trong comparator của `sort`, vì có thể phá tính bắc cầu. Cần xét riêng đường thẳng không hợp lệ, đoạn độ dài 0, điểm thẳng hàng và trường hợp tiếp xúc.

2.3 Quy hoạch động xác suất

Đặt $f_i(k)$ là xác suất có đúng k lần thành công sau i phép thử Bernoulli *độc lập*, với xác suất thành công ở lần i là p_i. Điều kiện đầu $f_0(0) = 1$, còn lại bằng 0. Tách theo kết quả lần thử cuối:

$$f_i(k) = (1 - p_i) f_{i-1}(k) + p_i f_{i-1}(k-1).$$

Duyệt k giảm dần để cập nhật tại chỗ; kết quả có $\sum_{k=0}^{n} f_n(k) = 1$. Thời gian $O(n^2)$, bộ nhớ $O(n)$.

Tổng quát hơn, nếu trạng thái chuyển theo đồ thị có hướng không chu trình, $\Pr(v) \mathrel{+}= \Pr(u)p(u,v)$ theo thứ tự tô pô. Tổng xác suất các cạnh đi ra trạng thái chưa kết thúc phải bằng 1. Nếu có chu trình, không được dùng một lượt tô pô.

2.4 Quy hoạch động và phương trình kỳ vọng

Gọi E_s là tổng chi phí kỳ vọng từ trạng thái s tới khi kết thúc; c_s là chi phí ngay tại s. Theo bước chuyển đầu tiên,

$$E_s = c_s + \sum_t p(s,t) E_t, \qquad E_{\text{terminal}} = 0.$$

Trên đồ thị không chu trình, tính E_s theo thứ tự tô pô ngược. Nếu chỉ có cạnh tự quay lại s với xác suất $p(s,s)$, chuyển vế:

$$E_s = \frac{c_s + \sum_{t \neq s} p(s,t) E_t}{1 - p(s,s)} \quad \text{khi } p(s,s) < 1.$$

Ví dụ mỗi lần thử thành công với xác suất $q > 0$, số lần thử kỳ vọng đến thành công thỏa $E = 1 + (1-q)E$, nên $E = 1/q$.

Với chu trình giữa nhiều trạng thái, lập hệ tuyến tính $(I - P)E = c$ trên các trạng thái chưa kết thúc. Trước khi giải, cần xét khả năng không bao giờ tới trạng thái kết thúc: nếu điều này có xác suất dương và chi phí mỗi bước bị chặn dưới bởi một hằng số dương, kỳ vọng là vô hạn. Không chia cho $1 - p(s,s)$ nếu vẫn còn phụ thuộc vòng giữa các trạng thái khác.

Mẫu C++17. Hàm thứ nhất tính phân phối số lần thành công; hàm thứ hai tính kỳ vọng trên đồ thị không chu trình. `order` là một thứ tự tô pô hợp lệ, `cost` là chi phí mỗi trạng thái, và trạng thái kết thúc có chi phí 0, không có cạnh đi ra. Trong `expectedDAG`, các trạng thái được đánh số $[0, n)$; `bernoulliCount` nhận `p[0]` cho phép thử thứ nhất.

```
using ld = long double;

// p[i]: xac suat thanh cong o phep thu i, cac phep thu doc lap.
vector<ld> bernoulliCount(const vector<ld>& p) {
    int n = (int)p.size();
    vector<ld> dp(n + 1);
    dp[0] = 1;
    for (int i = 0; i < n; ++i) {
        for (int k = i + 1; k >= 1; --k)
            dp[k] = dp[k] * (1 - p[i]) + dp[k - 1] * p[i];
        dp[0] *= 1 - p[i];
    }
    return dp;
}

// order: thu tu topo; moi canh u -> v di ve phia sau trong order.
// terminal: cost = 0, khong co canh di ra.
vector<ld> expectedDAG(
    const vector<vector<pair<int, ld>>>& edges,
    const vector<ld>& cost, const vector<int>& order
) {
    int n = (int)edges.size();
    vector<ld> e(n);
    for (int i = n - 1; i >= 0; --i) {
        int u = order[i];
        e[u] = cost[u];
        for (auto [v, prob] : edges[u]) e[u] += prob * e[v];
    }
    return e;
}
```

3 Lý thuyết trò chơi

3.1 Trạng thái thắng, thua và hòa

Xét trò chơi hai người, thông tin hoàn hảo, không ngẫu nhiên, cùng tập nước đi cho cả hai người; ai *không đi được* ở lượt mình thì thua (quy tắc chơi chuẩn). Với đồ thị trạng thái không chu trình, đặt $W(s)$ là trạng thái thắng của người sắp đi. Trạng thái không có nước đi là thua; trạng thái có ít nhất một cạnh tới trạng thái thua là thắng; nếu mọi cạnh tới trạng thái thắng thì là thua. Tính theo thứ tự topo ngược. Nếu cần giá trị điểm thay vì thắng/thua, dùng minimax với quy ước giá trị nhất quán theo người chơi.

3.2 Nim

Với các đống có kích thước $a_1, \ldots, a_k$, mỗi lượt giảm đúng một đống đi một lượng dương. Gọi $x = a_1 \oplus \cdots \oplus a_k$. Trạng thái thua khi và chỉ khi $x = 0$. Nếu $x = 0$, mọi nước đi đều làm XOR khác 0. Nếu $x \neq 0$, chọn bit cao nhất bật trong x, tìm đống a_i có bit ấy bật, và đổi nó thành $a_i' = a_i \oplus x < a_i$ để XOR mới bằng 0. Kết luận này dùng quy tắc chơi chuẩn. Với misère Nim (ai lấy viên cuối cùng thì thua): nếu mọi đống khác rỗng đều có kích thước 1, người sắp đi thua khi số đống đó lẻ; nếu còn đống lớn hơn 1, tiêu chuẩn thắng vẫn là XOR khác 0.

3.3 Số Grundy và mex

Với trò chơi vô tư hữu hạn, không chu trình, định nghĩa

$$g(s) = \operatorname{mex}\{g(t) : s \to t\}, \qquad \operatorname{mex}(S) = \min\{z \geq 0 : z \notin S\}.$$

Trạng thái hết nước đi có $g = 0$; $g(s) = 0$ đúng khi s thua. Nếu một lượt chỉ được đi trong *một* trong nhiều trò chơi độc lập, định lý Sprague–Grundy cho $g_{\text{tổng}} = g_1 \oplus \cdots \oplus g_k$. Không dùng XOR này khi một nước đi có thể thay đổi đồng thời nhiều thành phần, hoặc khi luật chơi là misère mà chưa biến đổi riêng.

3.4 Trò chơi trên đồ thị có chu trình

Trên đồ thị hữu hạn có chu trình, quy hoạch động topo không còn hợp lệ. Đánh dấu các đỉnh hết nước là thua, rồi lan ngược: tiền nhiệm của đỉnh thua là thắng; tiền nhiệm mà *mọi* nước đi đều tới đỉnh thắng là thua. Các đỉnh chưa được đánh dấu có thể duy trì ván đấu mãi mãi, nên là hòa nếu chơi tối ưu theo quy ước không kết thúc là hòa. Cần xác định rõ luật của đề: trò chơi có giới hạn lượt hoặc có trạng thái lặp bị xử thua sẽ cho kết quả khác.

Mẫu C++17. `grundyDAG` nhận thứ tự topo; `gameOutcome` trả 0 thua, 1 thắng, 2 hòa. Hai hàm nhận đỉnh $[0, n)$.

```
// order la thu tu topo cua do thi trang thai khong chu trinh.
vector<int> grundyDAG(const vector<vector<int>>& g, const vector<int>& order) {
    int n = (int)g.size();
    vector<int> ans(n);
    for (int i = n - 1; i >= 0; --i) {
        int u = order[i];
        vector<char> seen(g[u].size() + 1);
        for (int v : g[u]) if (ans[v] < (int)seen.size()) seen[ans[v]] = 1;
        while (seen[ans[u]]) ++ans[u];
    }
    return ans;
}

// 0: thua, 1: thang, 2: hoa neu cho phep choi vo han.
vector<int> gameOutcome(const vector<vector<int>>& g) {
    int n = (int)g.size();
    vector<vector<int>> rev(n);
    vector<int> remain(n), ans(n, 2);
    queue<int> q;
    for (int u = 0; u < n; ++u) {
        remain[u] = (int)g[u].size();
```

```
        for (int v : g[u]) rev[v].push_back(u);
        if (g[u].empty()) ans[u] = 0, q.push(u);
    }
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : rev[v]) {
            if (ans[u] != 2) continue;
            if (ans[v] == 0) ans[u] = 1, q.push(u);
            else if (--remain[u] == 0) ans[u] = 0, q.push(u);
        }
    }
    return ans;
}
```

4 Tổ hợp

4.1 Tổ hợp và đếm cách phân phối

Với $0 \le k \le n$, số cách chọn k trong n phần tử phân biệt, không xét thứ tự, là

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}, \qquad \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

Quy ước $\binom{n}{k} = 0$ nếu $k < 0$ hoặc $k > n$. Số hoán vị của đa tập có n_i phần tử loại i và $\sum_i n_i = n$ là $n!/\prod_i n_i!$.

Số nghiệm nguyên không âm của $x_1 + \cdots + x_m = s$ là $\binom{s+m-1}{m-1}$ khi $m \ge 1, s \ge 0$ (đặt $m-1$ vách ngăn). Nếu $x_i \ge l_i$, thay $y_i = x_i - l_i$. Nếu thêm $x_i \le u_i$, đặt $t = s - \sum_i l_i$, $b_i = u_i - l_i \ge 0$. Số nghiệm là

$$\sum_{J \subseteq [m]} (-1)^{|J|} \binom{t - \sum_{j \in J}(b_j + 1) + m - 1}{m-1}.$$

Đây là bao hàm–loại trừ trên các điều kiện xấu $y_i > b_i$. Nếu $t < 0$, đáp án bằng 0.

4.2 Số Catalan

Đặt $C_0 = 1$. Tách dãy ngoặc đúng theo cặp ngoặc đầu tiên:

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{1}{n+1}\binom{2n}{n}.$$

C_n đếm dãy ngoặc đúng n cặp, đường đi lưới từ $(0,0)$ tới (n,n) với mọi tiền tố giữ $y \le x$, và số cách tam giác hóa đa giác lồi $n+2$ đỉnh (với $n \ge 1$). Hiệu hai hệ số nhị thức suy ra từ nguyên lý phản xạ. Khi tính modulo M, không chia cho $n+1$ nếu nó không khả nghịch; dùng hiệu hai hệ số nhị thức hoặc cách khác thích hợp.

4.3 Bao hàm–loại trừ

Với các tập hữu hạn $A_1, \ldots, A_k$,

$$\left|\bigcup_{i=1}^{k} A_i\right| = \sum_{\varnothing \ne J \subseteq [k]} (-1)^{|J|+1} \left|\bigcap_{j \in J} A_j\right|.$$

Một phần tử nằm trong đúng $r \ge 1$ tập được tính với hệ số $\sum_{j=1}^{r} (-1)^{j+1}\binom{r}{j} = 1$. Để đếm đối tượng không mắc điều kiện xấu nào, lấy toàn bộ trừ số đối tượng thuộc hợp. Phương pháp trực tiếp có 2^k giao, chỉ dùng khi số điều kiện đủ nhỏ hoặc có thể gom các tập con theo một đại lượng chung.

4.4 Định lý Burnside

Nhóm hữu hạn G tác động lên tập cấu hình hữu hạn X; hai cấu hình tương đương nếu một phép trong G biến cấu hình này thành cấu hình kia. Số lớp tương đương:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |\operatorname{Fix}(g)|, \qquad \operatorname{Fix}(g) = \{x : g(x) = x\}.$$

Đếm các cặp (g, x) với $g(x) = x$ theo g cho tổng ở tử; theo định lý quỹ đạo–ổn định, mỗi lớp đóng góp $|G|$ cặp. Nếu g hoán vị n vị trí và tô bằng q màu, cấu hình cố định phải có cùng màu trên mỗi chu trình, nên $|\operatorname{Fix}(g)| = q^{c(g)}$. Tô vòng n ô, chỉ đồng nhất theo phép quay, cho

$$\frac{1}{n} \sum_{r=0}^{n-1} q^{\gcd(n,r)}.$$

Nếu còn đồng nhất qua phép lật, phải cộng các phép lật vào nhóm. Khi tính modulo, phép chia cho $|G|$ bằng nghịch đảo chỉ hợp lệ nếu $\gcd(|G|, M) = 1$.

Hệ số nhị thức modulo số nguyên tố (C++17). Yêu cầu p nguyên tố và $0 \le N < p$; sau tiền xử lý $O(N)$, mỗi truy vấn $O(1)$. Hàm Catalan cần $2n \le N$.

```
struct Binomial {
    long long p;
    vector<long long> fact, invfact;
    long long mul(long long a, long long b) const {
        return (long long)((__int128)a * b % p);
    }
    long long power(long long a, long long e) const {
        long long r = 1;
        for (; e; e >>= 1, a = mul(a, a))
            if (e & 1) r = mul(r, a);
        return r;
    }
    // p nguyen to, 0 <= N < p.
    Binomial(int N, long long prime)
```

```
        : p(prime), fact(N + 1), invfact(N + 1) {
        fact[0] = 1;
        for (int i = 1; i <= N; ++i) fact[i] = mul(fact[i - 1], i);
        invfact[N] = power(fact[N], p - 2);
        for (int i = N; i >= 1; --i)
            invfact[i - 1] = mul(invfact[i], i);
    }
    long long choose(int n, int k) const {
        if (k < 0 || k > n) return 0;
        return mul(fact[n], mul(invfact[k], invfact[n - k]));
    }
    long long catalan(int n) const {
        long long r = choose(2 * n, n) - choose(2 * n, n + 1);
        return r < 0 ? r + p : r;
    }
};
```

4.5 Nội suy Lagrange

Trên trường $\mathbb{F}_p$ với p nguyên tố, cho $k+1$ điểm (x_i, y_i) có các x_i đôi một khác nhau modulo p. Đa thức duy nhất P bậc không quá k đi qua các điểm đó là

$$P(t) = \sum_{i=0}^{k} y_i \prod_{\substack{0 \le j \le k \\ j \ne i}} \frac{t - x_j}{x_i - x_j} \pmod p.$$

Thật vậy, hạng ứng với i bằng y_i tại $t = x_i$ và bằng 0 tại các mốc còn lại; hiệu của hai đa thức bậc không quá k có $k+1$ nghiệm nên bằng 0. Phép chia modulo chỉ hợp lệ khi mẫu khác 0 modulo p; tính nghịch đảo $a^{p-2} \bmod p$.

Trường hợp thường gặp: biết $y_i = P(i)$ với $0 \le i \le k < p$. Khi đó

$$\prod_{j \ne i} (i - j) = (-1)^{k-i} i!(k-i)!.$$

Tích tiền tố và hậu tố của $(t - j)$ cho mọi tử số trong $O(k)$; nghịch đảo các giai thừa lấy từ một lần lũy thừa nhanh. Mỗi lần tính $P(t)$ tốn $O(k + \log p)$ thời gian và $O(k)$ bộ nhớ. Nếu $t \bmod p$ trùng một mốc, trả ngay y_i. Điều kiện bậc không quá k là thiết yếu: chỉ $k+1$ giá trị không xác định được đa thức bậc cao hơn.

```
// p nguyen to; y[i] = P(i), 0 <= i <= k < p.
long long lagrange(const vector<long long>& y, long long t, long long p) {
    assert(p >= 2 && !y.empty() && (long long)y.size() <= p);
    auto norm = [&](long long a) { a %= p; return a < 0 ? a + p : a; };
    auto mul = [&](long long a, long long b) {
        return (long long)((__int128)a * b % p);
    };
    auto power = [&](long long a, long long e) {
        long long r = 1;
        for (; e; e >>= 1, a = mul(a, a))
            if (e & 1) r = mul(r, a);
        return r;
    };
    int k = (int)y.size() - 1;
    t = norm(t);
    if (t <= k) return norm(y[(int)t]);

    vector<long long> pref(k + 2, 1), suf(k + 2, 1), invfact(k + 1);
    long long fact = 1;
    for (int i = 1; i <= k; ++i) fact = mul(fact, i);
    invfact[k] = power(fact, p - 2);
    for (int i = k; i >= 1; --i) invfact[i - 1] = mul(invfact[i], i);
    for (int i = 0; i <= k; ++i) pref[i + 1] = mul(pref[i], norm(t - i));
    for (int i = k; i >= 0; --i) suf[i] = mul(suf[i + 1], norm(t - i));

    long long ans = 0;
    for (int i = 0; i <= k; ++i) {
        long long term = mul(norm(y[i]), mul(pref[i], suf[i + 1]));
        term = mul(term, mul(invfact[i], invfact[k - i]));
        if ((k - i) & 1) term = term ? p - term : 0;
        ans = (long long)(((__int128)ans + term) % p);
    }
    return ans;
}
```

5 Số học

5.1 Euclid mở rộng và nghịch đảo modulo

Với $a, b > 0$, thuật toán Euclid cho $\gcd(a, b)$ bằng cách lặp $(a, b) \leftarrow (b, a \bmod b)$; độ phức tạp $O(\log \min(a, b))$. Euclid mở rộng còn tìm x, y sao cho

$$ax + by = \gcd(a, b).$$

Nếu $g = \gcd(a, m) = 1$, hệ số x thỏa $ax \equiv 1 \pmod m$, nên $x \bmod m$ là nghịch đảo của a modulo m. Nghịch đảo tồn tại *khi và chỉ khi* $g = 1$; không cần m nguyên tố. Mọi phép chia modulo m phải nhân với nghịch đảo của mẫu, và chỉ hợp lệ khi nghịch đảo tồn tại. Nếu m nguyên tố và $a \not\equiv 0 \pmod m$, cũng có thể dùng $a^{m-2} \bmod m$ theo định lý Fermat.

Phương trình $ax \equiv b \pmod m$ có nghiệm khi và chỉ khi $g = \gcd(a, m)$ chia hết b. Khi đó chia cả ba số cho g, tìm một nghiệm modulo m/g, rồi có đúng g nghiệm khác nhau modulo m.

5.2 Hệ phương trình đồng dư và CRT

Xét $x \equiv r_1 \pmod{m_1}$ và $x \equiv r_2 \pmod{m_2}$, với $m_1, m_2 > 0$. Đặt $g = \gcd(m_1, m_2)$. Hệ có nghiệm *khi và chỉ khi* $g \mid (r_2 - r_1)$. Viết $x = r_1 + m_1 t$ và chia đồng dư thứ hai cho g:

$$\frac{m_1}{g} t \equiv \frac{r_2 - r_1}{g} \pmod{\frac{m_2}{g}}.$$

Hai hệ số $m_1/g, m_2/g$ nguyên tố cùng nhau, nên tìm được t bằng Euclid mở rộng. Mọi nghiệm tạo một lớp modulo $\operatorname{lcm}(m_1, m_2)$; ghép lần lượt nhiều phương trình. Nếu các modulo đôi một nguyên tố cùng nhau, điều kiện tồn tại luôn đúng và modulo kết quả là tích các modulo. Kiểm tra tràn khi nhân modulo; chuẩn hóa phần dư về $[0, m)$, kể cả khi dữ liệu ban đầu âm.

Mẫu C++17. `inverseMod` trả rỗng nếu không có nghịch đảo; `mergeCRT` trả rỗng nếu hệ vô nghiệm, và ném lỗi nếu modulo kết quả vượt `long long`. CRT cần modulo dương; `inverseMod` chỉ xét $m \geq 2$ và trả rỗng nếu $m \leq 1$. `extendedGcd` nhận $a, b \geq 0$, không đồng thời bằng 0.

```
// a,b >= 0; khong dong thoi bang 0.
long long extendedGcd(long long a, long long b, long long& x, long long& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    long long x1, y1;
    long long g = extendedGcd(b, a % b, x1, y1);
    x = y1;
    y = (long long)((__int128)x1 - (__int128)(a / b) * y1);
    return g;
}
long long modNorm(long long a, long long m) {
    a %= m;
    if (a < 0) a += m;
    return a;
}
optional<long long> inverseMod(long long a, long long m) {
    if (m <= 1) return nullopt;
    long long x, y;
    if (extendedGcd(modNorm(a, m), m, x, y) != 1) return nullopt;
    return modNorm(x, m);
}
// r1 (mod m1), r2 (mod m2); m1,m2 > 0.
optional<pair<long long, long long>> mergeCRT(
    long long r1, long long m1, long long r2, long long m2
) {
    r1 = modNorm(r1, m1);
    r2 = modNorm(r2, m2);
    long long x, y, g = extendedGcd(m1, m2, x, y);
    long long diff = r2 - r1;
    if (diff % g) return nullopt;
    long long mod = m2 / g;
    __int128 t = (__int128)(diff / g) * x % mod;
    if (t < 0) t += mod;
    __int128 lcm = (__int128)(m1 / g) * m2;
    if (lcm > LLONG_MAX) throw overflow_error("CRT modulus overflow");
    __int128 r = (r1 + (__int128)m1 * t) % lcm;
    return pair<long long, long long>{(long long)r, (long long)lcm};
}
```

5.3 Số nguyên lớn trong C++ và Python

Nếu kết quả vượt `long long` và không thể chỉ giữ modulo, Python có kiểu `int` độ dài tùy ý; các phép `+,-,*,//,%`, `pow(a,b,m)` và `math.comb(n,k)` dùng trực tiếp. Trong C++, có thể dùng `boost::multiprecision::cpp_int`; ví dụ `cpp_int x = 1; x <<= 200;` tạo số 2^{200}. Kiểu này hỗ trợ cộng, trừ, nhân, chia và nhập xuất như số nguyên thông thường, nhưng chi phí thời gian và bộ nhớ lớn hơn kiểu máy. Phải kiểm tra quy định ngôn ngữ/thư viện của kỳ thi trước khi dựa vào Boost hoặc Python. Nếu chỉ cần tránh tràn *trung gian*, dùng `__int128` rồi ép về kiểu nhỏ sau khi đã kiểm tra giới hạn.

BigNum tự cài bằng chuỗi (C++17). Lưu trị tuyệt đối dạng thập phân và một bit dấu; chuẩn hóa số 0 thành `val="0", negative=false`. So sánh độ lớn theo số chữ số, sau đó so từ điển. Cộng/trừ xử lý dấu rồi cộng/trừ trị tuyệt đối; nhân từng chữ số, nhớ sang hàng kế tiếp. Cộng, trừ tốn $O(n)$; nhân kiểu cơ bản tốn $O(nm)$ với hai toán hạng dài n, m chữ số. Mẫu hỗ trợ nhập, in, so sánh, `+`, `-`, `*`; không có phép chia. Khi được phép, `cpp_int` thường tiện hơn.

```
struct BigNum {
    string val;
    bool negative = false;

    BigNum(string s = "0") {
        assert(!s.empty());
        if (s[0] == '-' || s[0] == '+') {
            negative = (s[0] == '-');
            s.erase(0, 1);
        }
        assert(!s.empty());
        assert(all_of(s.begin(), s.end(), [](char c) { return '0' <= c && c <= '9'; }));
        auto p = s.find_first_not_of('0');
        val = (p == string::npos ? "0" : s.substr(p));
        if (val == "0") negative = false;
    }

    void print(bool endline = false) const {
        if (negative) cout << '-';
        cout << val;
        if (endline) cout << '\n';
    }

    void input() {
        string s;
        cin >> s;
        *this = BigNum(s);
    }

    static int cmpAbs(const string &a, const string &b) {
        if (a.size() != b.size()) return a.size() < b.size() ? -1 : 1;
        return a == b ? 0 : (a < b ? -1 : 1);
    }

    static string addString(const string &a, const string &b) {
        string res;
```

```
        int carry = 0;
        for (int i = (int)a.size() - 1, j = (int)b.size() - 1;
             i >= 0 || j >= 0 || carry; --i, --j) {
            int cur = carry + (i >= 0 ? a[i] - '0' : 0)
                            + (j >= 0 ? b[j] - '0' : 0);
            res += char('0' + cur % 10);
            carry = cur / 10;
        }
        reverse(res.begin(), res.end());
        return res;
    }

    // Precondition: a >= b as nonnegative decimal integers.
    static string subString(const string &a, const string &b) {
        string res;
        int borrow = 0;
        for (int i = (int)a.size() - 1, j = (int)b.size() - 1;
             i >= 0; --i, --j) {
            int cur = a[i] - '0' - borrow - (j >= 0 ? b[j] - '0' : 0);
            borrow = (cur < 0);
            if (borrow) cur += 10;
            res += char('0' + cur);
        }
        while (res.size() > 1 && res.back() == '0') res.pop_back();
        reverse(res.begin(), res.end());
        return res;
    }

    bool operator < (const BigNum &b) const {
        if (negative != b.negative) return negative;
        int c = cmpAbs(val, b.val);
        return negative ? c > 0 : c < 0;
    }
    bool operator > (const BigNum &b) const { return b < *this; }
    bool operator <= (const BigNum &b) const { return !(b < *this); }
    bool operator >= (const BigNum &b) const { return !(*this < b); }
    bool operator == (const BigNum &b) const {
        return negative == b.negative && val == b.val;
    }
    bool operator != (const BigNum &b) const { return !(*this == b); }

    BigNum operator + (const BigNum &b) const {
        BigNum res;
        if (negative == b.negative) {
            res.val = addString(val, b.val);
            res.negative = negative;
        } else {
            int c = cmpAbs(val, b.val);
            if (c == 0) return BigNum("0");
            res.val = c > 0 ? subString(val, b.val) : subString(b.val, val);
            res.negative = c > 0 ? negative : b.negative;
        }
        if (res.val == "0") res.negative = false;
        return res;
    }

    BigNum operator - (const BigNum &b) const {
        BigNum opposite = b;
        if (opposite.val != "0") opposite.negative = !opposite.negative;
        return *this + opposite;
    }

    BigNum operator * (const BigNum &b) const {
        vector<int> digits(val.size() + b.val.size());
        for (int i = (int)val.size() - 1; i >= 0; --i)
            for (int j = (int)b.val.size() - 1; j >= 0; --j) {
                int p = i + j + 1;
                int cur = digits[p] + (val[i] - '0') * (b.val[j] - '0');
                digits[p] = cur % 10;
                digits[p - 1] += cur / 10;
            }
        string s;
        for (int x : digits) s += char('0' + x);
        BigNum res(s);
        if (res.val != "0") res.negative = negative != b.negative;
        return res;
    }
};
```

6 Đại số tuyến tính

6.1 Khử Gauss và số nghiệm

Với hệ $Ax = b$ có n phương trình, m ẩn, phép đổi hai hàng, nhân một hàng với số khác 0, hoặc cộng bội của hàng khác không đổi tập nghiệm. Khử từng cột: tìm hàng có hệ số trụ khác 0, đưa lên, chuẩn hóa rồi loại biến ở các hàng khác. Sau khử, nếu có hàng $0 = b_i$ với $b_i \neq 0$, hệ vô nghiệm. Nếu không, số biến tự do là $m - \text{rank}(A)$; hệ có nghiệm duy nhất khi hạng bằng m, và vô số nghiệm trên trường vô hạn khi hạng nhỏ hơn m. Với số thực, chọn trụ có trị tuyệt đối lớn nhất trong cột để giảm sai số; ε phụ thuộc thang dữ liệu. Nếu hệ có nghiệm vô hạn, mẫu code đặt biến tự do bằng 0 và trả về một nghiệm.

6.2 Khử Gauss trên GF(2)

Trên GF(2), cộng và trừ đều là XOR, nhân là AND. Biểu diễn mỗi hàng phương trình bằng bitmask các hệ số và một bit vế phải; khử bằng XOR hàng nên không có sai số số thực. Tiêu chuẩn vô nghiệm/hữu hạn nghiệm như trên: hàng hệ số bằng 0 mà vế phải bằng 1 là vô nghiệm; nếu có nghiệm, số nghiệm là $2^{m-\text{rank}(A)}$. Mẫu code hỗ trợ tối đa 64 ẩn và trả về một nghiệm với các biến tự do bằng 0. Trong hai mẫu code, ẩn thứ nhất có chỉ số 0 (bit 0 với GF(2)).

```
using ld = long double;
```

```
const ld GAUSS_EPS = 1e-12L;
struct GaussResult {
    int status; // 0: vo nghiem, 1: duy nhat, 2: vo so
    vector<ld> x;
};
// a la ma tran n x (m+1), cot cuoi la ve phai.
GaussResult gaussReal(vector<vector<ld>> a, int m) {
    int n = (int)a.size(), row = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && row < n; ++col) {
        int sel = row;
        for (int i = row + 1; i < n; ++i)
            if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
        if (abs(a[sel][col]) <= GAUSS_EPS) continue;
        swap(a[sel], a[row]);
        where[col] = row;
        ld div = a[row][col];
        for (int j = col; j <= m; ++j) a[row][j] /= div;
        for (int i = 0; i < n; ++i) if (i != row) {
            ld factor = a[i][col];
            for (int j = col; j <= m; ++j)
                a[i][j] -= factor * a[row][j];
        }
        ++row;
    }
    for (int i = 0; i < n; ++i) {
        bool zero = true;
        for (int j = 0; j < m; ++j)
            if (abs(a[i][j]) > GAUSS_EPS) zero = false;
        if (zero && abs(a[i][m]) > GAUSS_EPS) return {0, {}};
    }
    vector<ld> x(m);
    for (int j = 0; j < m; ++j)
        if (where[j] != -1) x[j] = a[where[j]][m];
    return {row == m ? 1 : 2, x};
}

struct Gauss2Result {
    int status, rank; // status: 0 vo nghiem, 1 duy nhat, 2 nhieu nghiem
    uint64_t x;       // nghiem voi cac bien tu do bang 0
};
// a[i] chua he so cua toi da 64 an; b[i] la ve phai 0/1.
Gauss2Result gaussGF2(vector<uint64_t> a, vector<int> b, int vars) {
    int n = (int)a.size(), row = 0;
    vector<int> where(vars, -1);
    for (int col = 0; col < vars && row < n; ++col) {
        int sel = row;
        while (sel < n && !(a[sel] >> col & 1ULL)) ++sel;
        if (sel == n) continue;
        swap(a[sel], a[row]); swap(b[sel], b[row]);
        where[col] = row;
        for (int i = 0; i < n; ++i) if (i != row && (a[i] >> col & 1ULL)) {
            a[i] ^= a[row];
            b[i] ^= b[row];
        }
        ++row;
    }
    for (int i = 0; i < n; ++i)
        if (a[i] == 0 && b[i]) return {0, row, 0};
    uint64_t x = 0;
    for (int col = 0; col < vars; ++col)
        if (where[col] != -1 && b[where[col]]) x |= 1ULL << col;
    return {row == vars ? 1 : 2, row, x};
}
```

6.3 Lũy thừa ma trận

Nếu trạng thái dạng vector cột thỏa $v_{t+1} = Mv_t$, thì $v_t = M^t v_0$. Nhân ma trận kết hợp nhưng không giao hoán, nên thứ tự nhân phải giữ đúng. Lũy thừa nhị phân dùng $M^0 = I$ và tính trong $O(d^3 \log t)$ thời gian, $O(d^2)$ bộ nhớ với ma trận vuông cấp d. Trong bài đếm đường đi dài đúng t trên đồ thị, $(M^t)_{uv}$ là số đường đi (walk) từ u tới v, nếu M_{uv} là số cạnh từ u tới v. Với truy hồi tuyến tính, đưa d giá trị gần nhất vào vector trạng thái rồi dựng ma trận chuyển tương ứng. Mẫu dưới lưu kích thước $m \times n$ trong `struct Matrix`; ô đầu tiên là `d[0][0]`. nhân được cả ma trận chữ nhật khi số cột trái bằng số hàng phải. Chỉ ma trận vuông mới được lũy thừa. Đổi `MATRIX_SIZE` và `MOD` theo đề, chuẩn hóa từng phần tử về $[0, \text{MOD})$ trước khi nhân. Tích trung gian dùng `__int128` để tránh tràn.

```
constexpr int MATRIX_SIZE = 64;
constexpr long long MOD = 1'000'000'007LL;
static_assert(MOD > 0);

// 0 <= a,b < MOD; tranh tran ngay ca khi MOD gan LLONG_MAX.
void add(long long &a, long long b) {
    if (a >= MOD - b) a -= MOD - b;
    else a += b;
}

struct Matrix {
    int m, n;
    long long d[MATRIX_SIZE][MATRIX_SIZE];

    Matrix(int _m = 0, int _n = 0) : m(_m), n(_n) {
        assert(0 <= m && m <= MATRIX_SIZE);
        assert(0 <= n && n <= MATRIX_SIZE);
        memset(d, 0, sizeof d);
    }

    Matrix operator * (const Matrix &other) const {
        assert(n == other.m);
        int x = m, y = n, z = other.n;
        Matrix ans(x, z);
        for (int i = 0; i < x; ++i)
            for (int j = 0; j < y; ++j)
                for (int k = 0; k < z; ++k)
```

```
                add(ans.d[i][k], (__int128)d[i][j] * other.d[j][k] % MOD);
        return ans;
    }

    Matrix operator ^ (long long k) const {
        assert(m == n && k >= 0);
        Matrix ans(n, n);
        for (int i = 0; i < n; ++i) ans.d[i][i] = 1 % MOD;
        Matrix mul = *this;
        for (; k; k >>= 1, mul = mul * mul)
            if (k & 1) ans = ans * mul;
        return ans;
    }
};
```

7 Luồng và lát cắt

7.1 Mô hình mạng luồng

Mạng có nguồn s, đích t, mỗi cạnh $e = (u, v)$ có sức chứa $c_e \geq 0$. Luồng f_e thỏa $0 \leq f_e \leq c_e$ và tại mọi đỉnh khác s, t, tổng luồng vào bằng tổng luồng ra. Giá trị luồng là luồng ròng ra khỏi s. Đồ thị thặng dư chứa cạnh thuận sức chứa $c_e - f_e$ và cạnh ngược sức chứa f_e; đi cạnh ngược nghĩa là hủy bớt luồng đã gửi. Mỗi cạnh được thêm vào code tạo một cặp cạnh thặng dư.

7.2 Luồng cực đại và lát cắt cực tiểu

Lát cắt $(S, V \setminus S)$ thỏa $s \in S, t \notin S$ có sức chứa $\sum_{u \in S, v \notin S} c(u, v)$. Mọi luồng có giá trị không vượt sức chứa mọi lát cắt. Khi không còn đường tăng từ s tới t trong đồ thị thặng dư, tập đỉnh còn tới được từ s cho một lát cắt có sức chứa đúng bằng giá trị luồng; đó là định lý max-flow min-cut. Nếu mọi sức chứa nguyên, tồn tại luồng cực đại nguyên.

7.3 Thuật toán Dinic

Lặp: BFS trên cạnh thặng dư dương để tạo đồ thị mức; DFS chỉ đi cạnh làm mức tăng 1 để đẩy một luồng chặn. Khi không tới được t, dừng. Mỗi pha xóa các đường ngắn nhất trong đồ thị thặng dư; số pha không quá $V - 1$, độ phức tạp tổng quát $O(V^2E)$. Mẫu code nhận đỉnh $[0, n)$, yêu cầu $s \neq t$; `maxflow` trả lượng luồng đẩy thêm trong lần gọi hiện tại, `minCutSide` trả các đỉnh ở phía nguồn của lát cắt tối tiểu sau khi chạy luồng cực đại. Tổng luồng phải vừa `long long`; DFS đệ quy có thể sâu $O(V)$.

```
struct Dinic {
    struct Edge { int to; long long cap; };
    int n;
    vector<Edge> e;
    vector<vector<int>> adj;
    vector<int> level, ptr;
    Dinic(int n) : n(n), adj(n), level(n), ptr(n) {}
    void addEdge(int u, int v, long long cap) {
        assert(cap >= 0);
        adj[u].push_back((int)e.size());
        e.push_back({v, cap});
        adj[v].push_back((int)e.size());
        e.push_back({u, 0});
    }
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        q.push(s); level[s] = 0;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : adj[u]) if (e[id].cap > 0 && level[e[id].to] == -1) {
                level[e[id].to] = level[u] + 1;
                q.push(e[id].to);
            }
        }
        return level[t] != -1;
    }
    long long dfs(int u, int t, long long pushed) {
        if (u == t || pushed == 0) return pushed;
        for (int& i = ptr[u]; i < (int)adj[u].size(); ++i) {
            int id = adj[u][i], v = e[id].to;
            if (level[v] != level[u] + 1 || e[id].cap == 0) continue;
            long long got = dfs(v, t, min(pushed, e[id].cap));
            if (got) {
                e[id].cap -= got;
                e[id ^ 1].cap += got;
                return got;
            }
        }
        return 0;
    }
    // Tra luong day THEM; dap an phai vua long long.
    long long maxflow(int s, int t) {
        assert(s != t);
        long long flow = 0;
        while (bfs(s, t)) {
```

```
            fill(ptr.begin(), ptr.end(), 0);
            while (long long got = dfs(s, t, LLONG_MAX)) {
                if (flow > LLONG_MAX - got) throw overflow_error("flow overflow");
                flow += got;
            }
        }
        return flow;
    }
    vector<char> minCutSide(int s) const {
        vector<char> vis(n);
        queue<int> q;
        q.push(s); vis[s] = 1;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int id : adj[u])
                if (e[id].cap > 0 && !vis[e[id].to])
                    vis[e[id].to] = 1, q.push(e[id].to);
        }
        return vis;
    }
};
```

7.4 Các mẫu dựng mạng từ đề bài

Đường đi không chung cạnh: đặt sức chứa 1 trên mỗi cạnh. Không chung đỉnh: tách v thành $v_{\text{in}} \to v_{\text{out}}$ sức chứa 1, trừ nguồn/đích nếu được dùng nhiều lần. Ghép cặp hai phía: $s \to L$, $L \to R$, $R \to t$, mọi cạnh sức chứa 1. Chọn dự án có lãi w_i: nếu $w_i > 0$, nối $s \to i$ sức chứa w_i; nếu $w_i < 0$, nối $i \to t$ sức chứa $-w_i$; điều kiện chọn i buộc chọn j tạo cạnh $i \to j$ sức chứa *lớn hơn tổng mọi lãi dương*. Đáp án là tổng lãi dương trừ sức chứa lát cắt tối thiểu. Tính ngưỡng vô cực theo dữ liệu, tránh tràn số.

7.5 Luồng có cận dưới

Với $l_e \le f_e \le u_e$, đặt $f_e = l_e + f'_e$, $0 \le f'_e \le u_e - l_e$. Với mỗi cạnh $u \to v$, cộng $d_v \mathrel{+}= l_e$, $d_u \mathrel{-}= l_e$. Thêm siêu nguồn S: nếu $d_v > 0$, thêm $S \to v$ sức chứa d_v; nếu $d_v < 0$, thêm $v \to T$ sức chứa $-d_v$. Tồn tại lưu thông hợp lệ khi và chỉ khi luồng cực đại từ S tới T bão hòa toàn bộ cạnh đi ra S. Để tìm luồng s–t không âm, thêm cạnh phụ $t \to s$ sức chứa đủ lớn trước khi kiểm tra. Nếu khả thi, gọi F_0 là luồng trên cạnh phụ. Muốn tối đa hóa: xóa các cạnh nối siêu nguồn/đích và cạnh phụ $t \to s$, *kể cả các cạnh ngược tương ứng*; giữ các cạnh dư của mạng gốc rồi đẩy thêm luồng từ s tới t. Đáp án là F_0 cộng lượng đẩy thêm, và $f_e = l_e + f'_e$.

7.6 Luồng chi phí cực tiểu

Mỗi cạnh còn có chi phí w_e cho một đơn vị luồng; cạnh ngược trong đồ thị thặng dư có chi phí $-w_e$. Để tìm luồng F đơn vị có chi phí nhỏ nhất, lặp đường ngắn nhất $s \to t$ trong đồ thị thặng dư và đẩy theo sức chứa nút cổ chai; dừng khi đủ F hoặc không còn đường. Với thế đỉnh π, chi phí giảm của cạnh là $w'(u,v) = w(u,v) + \pi_u - \pi_v$. Sau khi khởi tạo thế bằng Bellman–Ford, các chi phí giảm trên phần tới được không âm, nên có thể dùng Dijkstra mỗi pha. Mẫu dưới khởi tạo Bellman–Ford từ một siêu nguồn ngầm nối tới mọi đỉnh, và từ chối mạng dư ban đầu có chu trình chi phí âm, kể cả ở thành phần rời nguồn. Chu trình âm có sức chứa hữu hạn không đồng nghĩa đáp án $-\infty$, nhưng cần thuật toán xử lý lưu thông âm riêng. Mẫu code nhận đỉnh $[0, n)$, yêu cầu $s \ne t$, lượng cần gửi không âm. Hàm trả lượng đẩy thêm và chi phí của lượng đó; khi không đủ đường đi, lượng trả về có thể nhỏ hơn yêu cầu. Chi phí, khoảng cách và thế đỉnh dùng `__int128`; tổng chi phí phải vừa kiểu này. Với A lần tăng luồng, thời gian là $O(VE + AE \log V)$. Số pha có thể lớn; khi F rất lớn, cần phân tích giới hạn cụ thể trước khi chọn thuật toán này.

```
struct MinCostFlow {
    struct Edge { int to, rev; long long cap; __int128 cost; };
    int n;
    vector<vector<Edge>> g;
    MinCostFlow(int n) : n(n), g(n) {}
    void addEdge(int u, int v, long long cap, long long cost) {
        assert(cap >= 0);
        int i = (int)g[u].size(), j = (int)g[v].size();
        g[u].push_back({v, j + (u == v), cap, cost});
        g[v].push_back({u, i, 0, -(__int128)cost});
    }
    // s != t, need >= 0. Tong chi phi phai vua __int128.
    // Bao loi neu mang du ban dau co chu trinh am (ke ca o thanh phan roi).
    pair<long long, __int128> run(int s, int t, long long need = LLONG_MAX) {
        assert(s != t && need >= 0);
        const __int128 INF = (__int128)1 << 120;
        // Sieu nguon ngam noi toi moi dinh bang canh chi phi 0.
        vector<__int128> pot(n, 0);
        for (int it = 0; it < n; ++it) {
            bool changed = false;
            for (int u = 0; u < n; ++u)
                for (auto ed : g[u]) if (ed.cap > 0 && pot[u] + ed.cost < pot[ed.to]) {
                    pot[ed.to] = pot[u] + ed.cost;
                    changed = true;
                }
            if (!changed) break;
            if (it == n - 1) throw invalid_argument("negative cost cycle");
        }
        long long flow = 0;
        __int128 cost = 0;
        while (flow < need) {
            vector<__int128> dist(n, INF);
            vector<int> pv(n, -1), pe(n, -1);
            priority_queue<pair<__int128,int>, vector<pair<__int128,int>>,
                           greater<pair<__int128,int>>> pq;
            dist[s] = 0; pq.push({0, s});
            while (!pq.empty()) {
                auto [d, u] = pq.top(); pq.pop();
                if (d != dist[u]) continue;
                for (int i = 0; i < (int)g[u].size(); ++i) {
                    auto& ed = g[u][i];
                    if (ed.cap == 0) continue;
```

```
            __int128 nd = d + ed.cost + pot[u] - pot[ed.to];
            if (nd < dist[ed.to]) {
                dist[ed.to] = nd; pv[ed.to] = u; pe[ed.to] = i;
                pq.push({nd, ed.to});
            }
        }
    }
    if (dist[t] == INF) break;
    for (int v = 0; v < n; ++v)
        if (dist[v] < INF) pot[v] += dist[v];
    long long add = need - flow;
    for (int v = t; v != s; v = pv[v])
        add = min(add, g[pv[v]][pe[v]].cap);
    for (int v = t; v != s; v = pv[v]) {
        Edge& ed = g[pv[v]][pe[v]];
        ed.cap -= add;
        g[v][ed.rev].cap += add;
        cost += (__int128)add * ed.cost;
    }
    flow += add;
}
return {flow, cost};
}
};
```

8 Cặp ghép

8.1 Cặp ghép hai phía

Đồ thị hai phía có tập đỉnh L, R và chỉ có cạnh giữa hai phía. Cặp ghép là tập cạnh không chung đầu mút; cặp ghép cực đại theo *số cạnh* có lực lượng lớn nhất. Đừng nhầm với cặp ghép *tối đại* chỉ có nghĩa là không thêm ngay được một cạnh. Đường tăng bắt đầu ở đỉnh chưa ghép bên L, luân phiên qua cạnh chưa ghép và cạnh đã ghép, kết thúc ở đỉnh chưa ghép bên R. Đảo trạng thái các cạnh trên đường này làm kích thước tăng 1. Một cặp ghép là cực đại khi và chỉ khi không còn đường tăng (định lý Berge).

8.2 Đường tăng và thuật toán Hopcroft–Karp

Hopcroft–Karp dùng BFS tìm độ dài ngắn nhất của các đường tăng, rồi DFS lấy nhiều đường tăng cùng độ dài trong một pha. Độ phức tạp $O((V+E)\sqrt{V})$, phù hợp khi hai phía và số cạnh lớn. Mẫu code nhận đỉnh trái $[0, n_L)$, đỉnh phải $[0, n_R)$; gọi `maxMatching()` một lần sau khi thêm hết cạnh.

```
struct HopcroftKarp {
    int nl, nr;
    vector<vector<int>> adj;
    vector<int> ml, mr, dist;
    int distNil;
    HopcroftKarp(int nl, int nr)
        : nl(nl), nr(nr), adj(nl), ml(nl, -1), mr(nr, -1), dist(nl) {}
    void addEdge(int u, int v) { adj[u].push_back(v); }
    bool bfs() {
        queue<int> q;
        fill(dist.begin(), dist.end(), -1);
        for (int u = 0; u < nl; ++u)
            if (ml[u] == -1) dist[u] = 0, q.push(u);
        distNil = INT_MAX;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (dist[u] >= distNil) continue;
            for (int v : adj[u]) {
                int w = mr[v];
                if (w == -1) distNil = dist[u] + 1;
                else if (dist[w] == -1)
                    dist[w] = dist[u] + 1, q.push(w);
            }
        }
        return distNil != INT_MAX;
    }
    bool dfs(int u) {
        for (int v : adj[u]) {
            int w = mr[v];
            if ((w == -1 && dist[u] + 1 == distNil) ||
                (w != -1 && dist[w] == dist[u] + 1 && dfs(w))) {
                ml[u] = v; mr[v] = u;
                return true;
            }
        }
        dist[u] = -1;
        return false;
    }
    int maxMatching() {
        int ans = 0;
        while (bfs())
            for (int u = 0; u < nl; ++u)
                if (ml[u] == -1 && dfs(u)) ++ans;
        return ans;
    }
    pair<vector<int>, vector<int>> minVertexCover() const {
        vector<char> vl(nl), vr(nr);
        queue<int> q;
        for (int u = 0; u < nl; ++u)
```

```
            if (ml[u] == -1) vl[u] = 1, q.push(u);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) if (ml[u] != v && !vr[v]) {
                vr[v] = 1;
                int w = mr[v];
                if (w != -1 && !vl[w]) vl[w] = 1, q.push(w);
            }
        }
        vector<int> left, right;
        for (int u = 0; u < nl; ++u) if (!vl[u]) left.push_back(u);
        for (int v = 0; v < nr; ++v) if (vr[v]) right.push_back(v);
        return {left, right};
    }
};
```

8.3 Định lý König

Trong đồ thị hai phía, kích thước cặp ghép cực đại bằng kích thước phủ đỉnh cực tiểu. Sau khi tìm ghép cực đại, bắt đầu từ các đỉnh trái chưa ghép; đi theo cạnh *chưa ghép* từ trái sang phải, rồi cạnh *đã ghép* từ phải sang trái. Gọi tập đạt được là Z_L, Z_R. Phủ đỉnh cực tiểu là $(L \setminus Z_L) \cup Z_R$. Mẫu Hopcroft–Karp có hàm `minVertexCover()` trả hai danh sách đỉnh theo công thức đó.

8.4 Bài toán gán có trọng số

Với n người, $m \geq n$ việc, chi phí c_{ij}, cần gán mỗi người một việc khác nhau để tổng chi phí nhỏ nhất. Đây là bài toán gán *đầy đủ ở phía trái*; thuật toán Hungarian chạy $O(n^2 m)$, kể cả chi phí âm. Nếu một cặp bị cấm, không tự ý đặt chi phí vô cực rồi chấp nhận kết quả: phải kiểm tra có gán hợp lệ hay không. Hàm dưới nhận ma trận $n \times m$ có mọi ô hữu hạn, trả tổng chi phí và cột được gán cho từng hàng. Trong vector đầu vào và kết quả, hàng/cột đánh số từ 0. Các chi phí đầu vào là `long long`, còn tổng chi phí và các biến trung gian dùng `__int128` để tránh tràn. Bài toán tìm tổng lớn nhất có thể đổi dấu mọi chi phí rồi lấy đối dấu kết quả, chú ý tràn với giá trị cực tiểu của kiểu số.

```
// Ma tran chi phi n x m, n <= m, tat ca cac o huu han.
pair<__int128, vector<int>> hungarian(const vector<vector<long long>>& a) {
    int n = (int)a.size();
    if (n == 0) return {0, {}};
    int m = (int)a[0].size();
    assert(n <= m);
    for (const auto& row : a) assert((int)row.size() == m);
    const __int128 INF = (__int128)1 << 120;
    vector<__int128> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    for (int i = 1; i <= n; ++i) {
        p[0] = i;
        int j0 = 0;
        vector<__int128> minv(m + 1, INF);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            __int128 delta = INF;
            for (int j = 1; j <= m; ++j) if (!used[j]) {
                __int128 cur = (__int128)a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            for (int j = 0; j <= m; ++j) {
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            }
            j0 = j1;
        } while (p[j0] != 0);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    vector<int> assignment(n);
    for (int j = 1; j <= m; ++j)
        if (p[j]) assignment[p[j] - 1] = j - 1;
    __int128 cost = 0;
    for (int i = 0; i < n; ++i) cost += a[i][assignment[i]];
    return {cost, assignment};
}
```

9 Logic trên đồ thị

9.1 Mệnh đề và đồ thị kéo theo

Trong 2-SAT, mỗi điều kiện là một mệnh đề có tối đa hai literal, ví dụ $(a \vee b)$. Một literal là biến x hoặc phủ định $\neg x$. Mệnh đề tương đương hai kéo theo $\neg a \Rightarrow b$ và $\neg b \Rightarrow a$. Biểu diễn

mỗi literal bằng một đỉnh; phủ định là đỉnh ghép cặp. Một đường đi $p \rightsquigarrow q$ nghĩa là nếu chọn p đúng thì buộc q đúng. Với một yêu cầu đơn a, thêm $(a \vee a)$. Khi thêm yêu cầu $a \Rightarrow b$, phải thêm cả cạnh $a \to b$ và cạnh phản đảo $\neg b \to \neg a$; hàm `implication(a,b)` bên dưới thực hiện đủ hai cạnh. Với yêu cầu $a \Leftrightarrow b$, thêm hai chiều kéo theo; với yêu cầu đúng đúng một trong hai, thêm $(a \vee b) \wedge (\neg a \vee \neg b)$.

9.2 Bài toán 2-SAT

Tạo đồ thị kéo theo $2n$ đỉnh, tìm các thành phần liên thông mạnh (SCC). Công thức có nghiệm khi và chỉ khi không có biến x nào mà x và $\neg x$ cùng SCC: nếu cùng SCC, chọn một giá trị sẽ kéo theo phủ định của chính nó. Chiều ngược lại suy ra bằng cách xét DAG các SCC theo thứ tự tô pô và chọn một trong mỗi cặp SCC phủ định. Độ phức tạp $O(n+m)$ với m mệnh đề. Ràng buộc có ba literal trở lên không được đưa thẳng vào mô hình 2-SAT; SAT tổng quát khó hơn về bản chất.

9.3 Dựng và đọc một nghiệm

Trong mẫu code, biến đánh số $i \in [0, n)$; dùng đỉnh $2i$ cho $\neg x_i$, $2i+1$ cho x_i, nên phủ định một literal là XOR với 1. Kosaraju tạo số SCC tăng dần theo thứ tự tô pô của DAG SCC; đặt $x_i = [\text{comp}(x_i) > \text{comp}(\neg x_i)]$. Quy ước so sánh có thể đảo nếu đổi cách đánh số SCC, vì thế nên kiểm bằng mệnh đề đơn (x) và $(\neg x)$. `solve()` trả rỗng khi vô nghiệm, ngược lại là vector 0/1 của các biến.

```
struct TwoSAT {
    int n;
    vector<vector<int>> g, rg;
    TwoSAT(int n) : n(n), g(2*n), rg(2*n) {}
    int lit(int var, bool value) const { return 2*var + (int)value; }
    void addEdge(int a, int b) {
        g[a].push_back(b);
        rg[b].push_back(a);
    }
    // Menh de a => b can ca canh phan dao !b => !a.
    void implication(int a, int b) {
        addEdge(a, b);
        addEdge(b ^ 1, a ^ 1);
    }
    void either(int a, int b) {
        implication(a ^ 1, b);
    }
    optional<vector<int>> solve() const {
        int N = 2*n;
        vector<char> seen(N);
        vector<int> order;
        for (int root = 0; root < N; ++root) if (!seen[root]) {
            vector<pair<int,int>> st{{root, 0}};
            seen[root] = 1;
            while (!st.empty()) {
                auto& [u, idx] = st.back();
                if (idx == (int)g[u].size()) {
                    order.push_back(u); st.pop_back();
                } else {
                    int v = g[u][idx++];
                    if (!seen[v]) seen[v] = 1, st.push_back({v, 0});
                }
            }
        }
        vector<int> comp(N, -1);
        int cc = 0;
        for (int z = N - 1; z >= 0; --z) {
            int root = order[z];
            if (comp[root] != -1) continue;
            vector<int> st{root};
            comp[root] = cc;
            while (!st.empty()) {
                int u = st.back(); st.pop_back();
                for (int v : rg[u]) if (comp[v] == -1)
                    comp[v] = cc, st.push_back(v);
            }
            ++cc;
        }
        vector<int> ans(n);
        for (int i = 0; i < n; ++i) {
            if (comp[2*i] == comp[2*i+1]) return nullopt;
            ans[i] = comp[2*i+1] > comp[2*i];
        }
        return ans;
    }
};
```

10 Xử lý chuỗi

10.1 Z-function

$Z[i]$ là độ dài tiền tố chung dài nhất của s và $s[i \dots n-1]$; quy ước $Z[0] = n$. Duy trì đoạn $[l, r)$ khớp với tiền tố. Nếu $i < r$, lấy cận dưới $\min(r - i, Z[i - l])$, rồi so sánh thêm; nếu vượt

r, cập nhật đoạn. Mỗi lần so sánh mở rộng làm r tăng nên tổng thời gian $O(n)$. Để tìm mẫu p trong văn bản t, tính Z của $p + \# + t$ với ký tự phân cách không có trong cả hai; vị trí có Z ít nhất $|p|$ là một lần khớp. Nếu bảng chữ không có ký tự phân cách, dùng chuỗi số nguyên với giá trị sentinel riêng.

10.2 Thuật toán Manacher

Đặt $d_1[i]$ là số bán kính đối xứng lẻ quanh ký tự i: độ dài dài nhất là $2d_1[i]-1$. Đặt $d_2[i]$ là số bán kính đối xứng chẵn với tâm giữa $i-1$ và i: độ dài dài nhất là $2d_2[i]$. Duy trì palindrome phải nhất $[l, r]$, dùng tâm đối xứng của i để khởi tạo bán kính rồi mở rộng. Tổng thời gian $O(n)$. Số xâu con đối xứng, tính cả vị trí trùng nội dung, là $\sum_i (d_1[i] + d_2[i])$. Mẫu code trả hai mảng bán kính này.

10.3 Trie

Trie lưu các tiền tố thành đỉnh. Với bảng chữ cố định, mỗi cạnh gắn một ký tự; chèn một xâu dài m mất $O(m)$. Đếm số xâu đi qua mỗi đỉnh để trả số xâu có một tiền tố, và đếm số xâu kết thúc để phân biệt tiền tố với từ hoàn chỉnh. Mẫu dưới chỉ hỗ trợ chữ thường `a`–`z`, cho phép chèn trùng và không hỗ trợ xóa; số nút tối đa bằng 1+ tổng độ dài các xâu đã chèn.

```
vector<int> zFunction(const string& s) {
    int n = (int)s.size();
    vector<int> z(n);
    if (n) z[0] = n;
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}

// d1[i]: so ban kinh le; d2[i]: so ban kinh chan.
pair<vector<int>, vector<int>> manacher(const string& s) {
    int n = (int)s.size();
    vector<int> d1(n), d2(n);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = i > r ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n &&
               s[i - k - 1] == s[i + k]) ++k;
        d2[i] = k--;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
    return {d1, d2};
}

struct Trie {
    struct Node {
        array<int, 26> next;
        int pass = 0, terminal = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> t{Node()};
    void insert(const string& s) {
        int u = 0;
        ++t[u].pass;
        for (char ch : s) {
            int c = ch - 'a';
            if (t[u].next[c] == -1) {
                t[u].next[c] = (int)t.size();
                t.emplace_back();
            }
            u = t[u].next[c];
            ++t[u].pass;
        }
        ++t[u].terminal;
    }
    int countPrefix(const string& s) const {
        int u = 0;
        for (char ch : s) {
            u = t[u].next[ch - 'a'];
            if (u == -1) return 0;
        }
        return t[u].pass;
    }
    int countWord(const string& s) const {
        int u = 0;
        for (char ch : s) {
            u = t[u].next[ch - 'a'];
            if (u == -1) return 0;
        }
        return t[u].terminal;
    }
};
```

10.4 Thuật toán Aho–Corasick

Xây Trie của nhiều mẫu, rồi BFS tạo suffix link: link của mỗi đỉnh trỏ tới hậu tố dài nhất vẫn là tiền tố của một mẫu. Khi thiếu cạnh ký tự c, chuyển theo link; các cạnh thiếu có thể điền sẵn để mỗi ký tự văn bản xử lý $O(1)$. Cộng số mẫu kết thúc ở các đỉnh trên chuỗi link để đếm mọi lần xuất hiện, kể cả mẫu trùng và các lần chồng nhau. Với bảng chữ cỡ σ, thời gian và bộ nhớ xây dựng $O(\sigma L)$ cho L nút; quét văn bản $O(|t|)$. Mẫu code đếm tổng số lần khớp, chỉ nhận chữ thường `a`–`z`; phải thêm hết mẫu trước khi gọi `build()` đúng một lần rồi mới truy vấn, và không thêm mẫu sau đó. Nếu thêm mẫu rỗng, nó khớp $|t| + 1$ lần, gồm cả trước ký tự đầu.

10.5 Suffix array

Mảng hậu tố SA sắp các vị trí i theo thứ tự từ điển của $s[i \dots n-1]$. Thuật toán doubling xếp theo cặp lớp tương đương của hai nửa độ dài 2^k; thêm sentinel nhỏ hơn mọi ký tự để biến thành sắp vòng, rồi bỏ sentinel. Dùng counting sort theo lớp cho $O(n \log n)$ thời gian và $O(n)$ bộ nhớ. Mẫu code nhận byte khác 0, trả các vị trí đánh số từ 0; xâu rỗng trả mảng rỗng. LCP có thể tính thêm bằng Kasai nếu bài cần so sánh nhiều hậu tố hoặc đếm xâu con phân biệt, nhưng không nên đưa cả bộ nếu bài chỉ cần thứ tự hậu tố.

```
struct AhoCorasick {
    struct Node {
        array<int, 26> next;
        int link = 0, out = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> t{Node()};
    bool built = false;
    void addPattern(const string& s) {
        assert(!built);
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (t[u].next[c] == -1) {
                t[u].next[c] = (int)t.size();
                t.emplace_back();
            }
            u = t[u].next[c];
        }
        ++t[u].out;
    }
    void build() {
        assert(!built);
        built = true;
        queue<int> q;
        for (int c = 0; c < 26; ++c) {
            int v = t[0].next[c];
            if (v == -1) t[0].next[c] = 0;
            else q.push(v);
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            t[u].out += t[t[u].link].out;
            for (int c = 0; c < 26; ++c) {
                int v = t[u].next[c];
                if (v == -1) t[u].next[c] = t[t[u].link].next[c];
                else {
                    t[v].link = t[t[u].link].next[c];
                    q.push(v);
                }
            }
        }
    }
    long long countMatches(const string& s) const {
        assert(built);
        int u = 0;
        long long ans = t[0].out; // Mau rong khop ca truoc ky tu dau.
        for (char ch : s) {
            u = t[u].next[ch - 'a'];
            ans += t[u].out;
        }
        return ans;
    }
};

// s khong chua byte 0; tra ve vi tri cac hau to theo thu tu tu dien.
vector<int> suffixArray(const string& s) {
    string a = s + char(0);
    int n = (int)a.size();
    vector<int> p(n), c(n), cnt(max(256, n));
    for (unsigned char ch : a) ++cnt[ch];
    for (int i = 1; i < 256; ++i) cnt[i] += cnt[i-1];
    for (int i = 0; i < n; ++i)
        p[--cnt[(unsigned char)a[i]]] = i;
    int classes = 1;
    for (int i = 1; i < n; ++i) {
        if (a[p[i]] != a[p[i-1]]) ++classes;
        c[p[i]] = classes - 1;
    }
    vector<int> pn(n), cn(n);
    for (int len = 1; len < n && classes < n; len <<= 1) {
        for (int i = 0; i < n; ++i)
            pn[i] = (p[i] - len + n) % n;
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int x : pn) ++cnt[c[x]];
        for (int i = 1; i < classes; ++i) cnt[i] += cnt[i-1];
        for (int i = n - 1; i >= 0; --i)
            p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
        int newClasses = 1;
        for (int i = 1; i < n; ++i) {
            pair<int,int> x{c[p[i]], c[(p[i] + len) % n]};
            pair<int,int> y{c[p[i-1]], c[(p[i-1] + len) % n]};
            if (x != y) ++newClasses;
            cn[p[i]] = newClasses - 1;
        }
        c.swap(cn);
        classes = newClasses;
    }
    return vector<int>(p.begin() + 1, p.end());
}
```

11 Tối ưu quy hoạch động

11.1 Convex Hull Trick

Nếu chuyển trạng thái có dạng

$$dp_i = \min_j \{m_j x_i + b_j\},$$

mỗi j là một đường thẳng $y = m_j x + b_j$; bài toán là thêm đường và hỏi giá trị nhỏ nhất tại x_i. Nếu độ dốc được thêm theo thứ tự *giảm nghiêm ngặt* và các truy vấn x *không giảm*, giữ bao dưới bằng deque/vector. Với ba đường liên tiếp a, b, c có $m_a > m_b > m_c$, đường b bị bỏ nếu giao điểm a, b không nằm trước giao điểm b, c:

$$(b_b - b_a)(m_b - m_c) \geq (b_c - b_b)(m_a - m_b).$$

Đây là so sánh chéo chính xác bằng số nguyên, không dùng số thực. Các tích của hiệu hệ số phải vừa `__int128`; kiểu này chưa đủ nếu các hiệu có thể trải hết miền 64 bit. Khi hai đường có cùng độ dốc, chỉ giữ đường có tung độ gốc nhỏ hơn. Truy vấn tăng x dùng con trỏ tiến; mỗi đường được thêm/xóa không quá một lần, tổng $O(n)$ sau khi đã có thứ tự nói trên. Phải thêm ít nhất một đường trước khi hỏi. Nếu độ dốc hoặc truy vấn không theo thứ tự, không dùng mẫu này mà đổi sang Li Chao hoặc cấu trúc khác.

```
struct MonotoneCHT {
    struct Line { long long m, b; };
    vector<Line> h;
    int ptr = 0;
    static __int128 value(Line a, long long x) {
        return (__int128)a.m * x + a.b;
    }
    static bool bad(Line a, Line b, Line c) {
        return ((__int128)b.b - a.b) * ((__int128)b.m - c.m) >=
               ((__int128)c.b - b.b) * ((__int128)a.m - b.m);
    }
    // Do doc m khong tang; truy van x khong giam.
    // Cac tich trong bad() phai vua __int128.
    void add(long long m, long long b) {
        assert(h.empty() || m <= h.back().m);
        Line line{m, b};
        if (!h.empty() && h.back().m == m) {
            if (h.back().b <= b) return;
            h.pop_back();
        }
        while (h.size() >= 2 && bad(h[h.size()-2], h.back(), line))
            h.pop_back();
        h.push_back(line);
        ptr = min(ptr, (int)h.size() - 1);
    }
    __int128 query(long long x) {
        assert(!h.empty());
        while (ptr + 1 < (int)h.size() &&
               value(h[ptr+1], x) <= value(h[ptr], x)) ++ptr;
        return value(h[ptr], x);
    }
};
```

11.2 Li Chao tree

Với miền truy vấn nguyên cố định $[L, R]$, Li Chao tree giữ ở mỗi nút một đường tối ưu tại điểm giữa. Khi thêm đường mới, so sánh ở đầu trái và điểm giữa, giữ đường tốt hơn tại giữa, rồi đẩy đường còn lại xuống đúng một nửa có thể thắng. Mỗi thao tác thêm/hỏi tốn $O(\log(R-L+1))$ và không cần thứ tự độ dốc hay truy vấn. Mẫu dưới hỗ trợ *min*, thêm cả đường trên toàn miền; phải hỏi $x \in [L, R]$. Giá trị hàm và đáp án dùng `__int128`; vẫn cần kiểm tra giới hạn tích $mx + b$.

```
struct LiChao {
    struct Line { long long m, b; };
    struct Node { Line line{0, 0}; int left = -1, right = -1; bool has = false; };
    long long lo, hi;
    vector<Node> t{Node()};
    static constexpr __int128 INF = numeric_limits<__int128>::max();
    LiChao(long long lo, long long hi) : lo(lo), hi(hi) { assert(lo <= hi); }
    static __int128 value(Line a, long long x) {
        return (__int128)a.m * x + a.b;
    }
    void add(Line line) { addRec(0, lo, hi, line); }
    void addRec(int u, long long l, long long r, Line line) {
        if (!t[u].has) {
            t[u].line = line; t[u].has = true; return;
        }
        long long mid = l + (long long)(((__int128)r - l) / 2);
        bool left = value(line, l) < value(t[u].line, l);
        bool middle = value(line, mid) < value(t[u].line, mid);
        if (middle) swap(line, t[u].line);
        if (l == r) return;
        if (left != middle) {
            if (t[u].left == -1) {
                t[u].left = (int)t.size(); t.emplace_back();
            }
            addRec(t[u].left, l, mid, line);
        } else {
            if (t[u].right == -1) {
                t[u].right = (int)t.size(); t.emplace_back();
            }
            addRec(t[u].right, mid + 1, r, line);
        }
    }
```

```
    __int128 query(long long x) const {
        assert(lo <= x && x <= hi);
        return queryRec(0, lo, hi, x);
    }
    __int128 queryRec(int u, long long l, long long r, long long x) const {
        if (u == -1) return INF;
        __int128 ans = t[u].has ? value(t[u].line, x) : INF;
        if (l == r) return ans;
        long long mid = l + (long long)(((__int128)r - l) / 2);
        int v = x <= mid ? t[u].left : t[u].right;
        return min(ans, x <= mid ? queryRec(v, l, mid, x)
                                 : queryRec(v, mid + 1, r, x));
    }
};
```

11.3 Các tối ưu khác

Với $D_k(j) = \min_{i<j}\{D_{k-1}(i) + C(i,j)\}$, nếu chỉ số tối ưu $\mathrm{opt}_k(j)$ không giảm theo j, có thể tính một hàng bằng chia để trị: tính điểm giữa với i trong khoảng tối ưu cho phép, rồi thu hẹp khoảng cho hai nửa. Chi phí $O(n \log n)$ lần gọi C mỗi hàng (nếu mỗi lần gọi C là $O(1)$); không tự suy ra tính đơn điệu chỉ từ vài ví dụ. Bất đẳng thức tứ giác/ma trận Monge là một điều kiện đủ thường gặp. Hàm mẫu giả thiết $D_{k-1}(i)$ đã biết và tính $j = 1, \ldots, n$; `prev[0]` là trạng thái trước vị trí đầu tiên. Giá trị vô cực được mã hóa bằng `LLONG_MAX/4`, mọi tổng hữu hạn phải nằm trong $(-\mathrm{INF}, \mathrm{INF})$. Khi hòa chọn chỉ số nhỏ nhất, kể cả khi mọi ứng viên đều vô cực; tính đơn điệu phải đúng với quy ước này.

```
// prev[0..n], cur[j] = min_{0<=i<j}(prev[i]+cost(i,j)).
// opt[j] la chi so nho nhat dat min, khong giam theo j.
// INF danh dau vo nghiem; moi gia tri huu han phai nam trong (-INF,INF).
vector<long long> divideConquerLayer(
    const vector<long long>& prev,
    const function<long long(int,int)>& cost
) {
    int n = (int)prev.size() - 1;
    const long long INF = LLONG_MAX / 4;
    vector<long long> cur(n + 1, INF);
    function<void(int,int,int,int)> solve = [&](int l, int r, int ql, int qr) {
        if (l > r) return;
        int mid = l + (r - l) / 2, bestI = ql;
        for (int i = ql; i <= min(qr, mid - 1); ++i) {
            if (prev[i] == INF) continue;
            long long val = prev[i] + cost(i, mid);
            if (val < cur[mid]) cur[mid] = val, bestI = i;
        }
        // Neu tat ca deu INF, ql van la vi tri min trai nhat.
        solve(l, mid - 1, ql, bestI);
        solve(mid + 1, r, bestI, qr);
    };
    solve(1, n, 0, n - 1);
    return cur;
}
```

Knuth áp dụng cho một dạng khác: $D(l,r) = \min_{l \le k < r}\{D(l,k) + D(k+1,r) + W(l,r)\}$. Nếu đã chứng minh được $\mathrm{opt}(l, r-1) \le \mathrm{opt}(l,r) \le \mathrm{opt}(l+1, r)$, chỉ thử k trong khoảng đó và giảm từ $O(n^3)$ xuống $O(n^2)$. Điều kiện này không đồng nghĩa với điều kiện của chia để trị. Khi chưa chứng minh được miền tối ưu, dùng DP gốc hoặc kiểm chứng giả thiết trên dữ liệu nhỏ trước khi tối ưu.

12 Tra cứu nhanh

12.1 Dấu hiệu nhận dạng dạng bài

Các dấu hiệu sau gợi ý hướng thử; luôn đối chiếu ràng buộc và chứng minh điều kiện áp dụng.

- **Tối ưu một giá trị số**: nếu mệnh đề đạt ít nhất x (hoặc không quá x) đơn điệu, thử tìm kiếm nhị phân trên đáp án. Ghi rõ hai biên và chiều đơn điệu.
- **Nhiều truy vấn, dữ liệu cố định**: cân nhắc tiền xử lý bằng prefix sum, sparse table, LCA hoặc xử lý ngoại tuyến. Nếu có cập nhật, xác định phép gộp rồi xét Fenwick/segment tree.
- **Phụ thuộc trên DAG**: topo rồi DP theo thứ tự topo. Nếu có chu trình, cần trạng thái hoặc lập luận khác.
- **Mỗi đỉnh có đúng một cạnh đi ra**: đồ thị hàm gồm các chu trình và cây đổ vào chúng; tách chu trình khi đếm hoặc nhảy xa.
- **Chọn/bỏ từng phần tử, n nhỏ**: xét bitmask DP hoặc meet-in-the-middle. 2^{20} xấp xỉ 10^6, nhưng còn phải tính số chuyển trạng thái và số bộ test.
- **Ghép cặp hợp lệ**: kiểm tra cấu trúc hai phía. Nếu có chi phí, xét bài toán gán; nếu có sức chứa, xét luồng.
- **Điều kiện kéo theo giữa các lựa chọn**: thử mô hình min-cut; cạnh sức chứa vô hạn cấm các cách chia không hợp lệ.
- **Quá trình ngẫu nhiên**: DP xác suất/kỳ vọng trên trạng thái; nếu có vòng lặp, lập hệ phương trình và xét khả năng không dừng.
- **Trò chơi hai người**: xác định thắng/thua từ các nước đi. Dùng Grundy khi trò chơi là tổng các phần độc lập phù hợp.
- **Tọa độ nguyên, hỏi hướng/diện tích**: ưu tiên tích có hướng số nguyên; giao điểm và đường tròn cần xét sai số, suy biến.

12.2 Gợi ý tư duy ad hoc và constructive

Tìm cấu trúc. Thử dữ liệu nhỏ và cực trị; thay số cụ thể bằng thứ tự, chẵn lẻ, phần dư hoặc quan hệ bằng nhau. Sắp xếp rồi xét đổi chỗ hai phần tử kề nhau: nếu đổi chỗ không làm đáp án xấu hơn, chứng minh bằng *exchange argument* để suy ra dạng nghiệm chuẩn. Với thao tác lặp, tìm bất biến hoặc thế năng nguyên bị chặn và thay đổi chặt để chứng minh không lặp vô hạn.

Dựng và chứng minh. Trước hết tìm điều kiện cần bằng bất biến, giới hạn tổng, chẵn lẻ, bậc hoặc đếm. Khi dựng từng bước, ghi rõ điều kiện còn giữ sau bước đó. Chứng minh luôn có bước tiếp theo, quá trình kết thúc, và cấu hình cuối thỏa *mọi* điều kiện. Nếu lập luận chỉ đúng từ một kích thước nào đó, xử lý riêng các trường hợp nhỏ. Với tham lam theo thứ tự từ điển, phải chứng minh lựa chọn hiện tại vẫn cho phép hoàn thành phần còn lại.

Tìm phản ví dụ. Thử $n = 0, 1, 2$ nếu hợp lệ; tất cả bằng nhau; tăng/giảm nghiêm ngặt; giá trị âm, 0, dương; các giá trị biên; nhiều nghiệm tối ưu. Với tham lam, tìm hai lựa chọn đầu tiên dẫn đến kết quả khác nhau. Vét cạn n nhỏ giúp bác bỏ giả thuyết, nhưng không thay thế chứng minh.

12.3 Công thức thường quên

- $\sum_{i=1}^{n} i = n(n+1)/2$; $\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6$; $\binom{n}{2} = n(n-1)/2$.
- $1 + r + \cdots + r^{n-1} = (r^n - 1)/(r-1)$ khi $r \neq 1$; với $r = 1$, tổng bằng n. Khi tính modulo, chỉ chia cho $r - 1$ nếu nghịch đảo modulo tồn tại.
- Với $P[0] = 0$, $P[i+1] = P[i] + a[i]$, tổng nửa mở $[l, r)$ là $P[r] - P[l]$. Độ dài giao hai đoạn nửa mở bằng $\max(0, \min(r_1, r_2) - \max(l_1, l_2))$.
- Với $a \geq 0$, $b > 0$: $\lceil a/b \rceil = \lfloor a/b \rfloor + [a \bmod b \neq 0]$. Cách viết $(a + b - 1)/b$ có thể tràn số nguyên.
- Với $a, b \neq 0$: $\operatorname{lcm}(a, b) = |a / \gcd(a, b) \cdot b|$; chia trước khi nhân nhưng vẫn kiểm tra tràn.
- Với $m > 0$, phần dư chuẩn hóa của a là $((a \bmod m) + m) \bmod m$ về mặt toán học. Trong C++, dùng `r=a%m; if(r<0) r+=m;` để tránh cộng thêm m khi không cần. Chia modulo m chỉ hợp lệ khi mẫu nguyên tố cùng nhau với m.
- Với $x > 0$, $x \,\&\, (-x)$ lấy bit 1 thấp nhất; $x \,\&\, (x-1)$ xóa bit 1 thấp nhất.
- $2S = |\sum_i \operatorname{cross}(P_i, P_{i+1})|$ cho đa giác đơn. Khi chỉ so sánh khoảng cách, dùng bình phương khoảng cách để tránh `sqrt`.
- $\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B)$. Với X nguyên không âm, $\mathbb{E}[X] = \sum_{k \geq 1} \Pr(X \geq k)$, kể cả khi tổng có thể vô hạn.

12.4 Checklist lỗi và trường hợp biên

Trước khi cài. Xác định đoạn đóng hay nửa mở, chỉ số từ 0 hay 1, trạng thái vô nghiệm, đồ thị có thể rời rạc/có cạnh song song/khuyên hay không. Ước lượng thời gian và bộ nhớ ở ràng buộc lớn nhất, kể cả nhiều test.

Trước khi nộp.

- Kiểm tra $n = 0$ hoặc 1 nếu hợp lệ, tập rỗng, mọi phần tử bằng nhau, không có cạnh, đồ thị không liên thông, đáp án bằng 0.
- Kiểm tra tràn ở tích, tổng đường đi, INF + w, tích ma trận, diện tích và lcm; ép kiểu `1LL` hoặc `__int128` *trước* khi nhân.
- Kiểm tra chia cho 0, modulo âm, nghịch đảo không tồn tại; số thực gần 0, đường song song/trùng, điểm trùng/thẳng hàng.
- Xóa trạng thái giữa các test: mảng đánh dấu, danh sách kề, queue, biến toàn cục, nén tọa độ và đáp án.
- Kiểm tra `i+1`, `i-1`, `lower_bound == end`, và phép trừ trên `size_t`.
- Nếu đệ quy sâu đến $O(n)$, xem giới hạn stack. Thử một test tự tính tay và vài test nhỏ ngẫu nhiên đối chiếu với vét cạn; khi sai, giữ seed và rút nhỏ phản ví dụ.

12.5 Cú pháp Python cho số nguyên lớn

Chỉ dùng nếu Python được phép và thời gian đủ. `int` không bị giới hạn 64 bit, nhưng phép toán trên số rất dài vẫn tốn thời gian.

- `a = int(input())`; với nhiều dòng, dùng `sys.stdin.buffer.readline`. Xuất bằng `print(a)`.
- `a // b` lấy sàn kể cả khi $a < 0$; `a % b` bằng 0 hoặc cùng dấu với b khi $b \neq 0$. `divmod(a,b)` trả về thương và dư.
- `pow(a,k,m)` tính $a^k \bmod m$ mà không tạo số a^k đầy đủ ($k \geq 0$, $m > 0$). `pow(a,-1,m)` tìm nghịch đảo nếu $\gcd(a, m) = 1$; nếu không sẽ phát sinh `ValueError`.
- `math.comb(n,k)`, `math.gcd(a,b)` và `math.isqrt(n)` cho tổ hợp, UCLN và $\lfloor\sqrt{n}\rfloor$ chính xác; `isqrt` cần $n \geq 0$. `comb`, `isqrt` và số mũ âm của `pow` với modulo cần Python 3.8 trở lên.
- Trên Python 3.11 trở lên, chuyển đổi số thập phân cực dài có thể vướng giới hạn chữ số. Khi đề thật sự cần, kiểm tra `sys.get_int_max_str_digits()` và có thể gọi `sys.set_int_max_str_digits(0)`. Kiểm tra phiên bản Python của máy chấm.

12.6 trick lỏ

pragma lấy trên codeforces ok

```
#pragma GCC optimize("Ofast,unroll-loops") // C: Ofast bat fast-math; unroll co the cham hon.
#pragma GCC target("avx2,popcnt,lzcnt,abm,bmi,bmi2,fma,tune=native") // C: x86, phu thuoc GCC/CPU; co
     the CE hoac loi lenh CPU.
#pragma GCC optimize(3) // B: O3; do toc do, khong dam bao nhanh hon O2.
#pragma GCC target("avx,avx2") // B: chi dung khi may cham x86 ho tro AVX2.
#pragma GCC optimize("Ofast") // C: hop le, nhung fast-math co the doi ket qua so thuc.
#pragma GCC optimize("inline") // C: cho phep inline, khong ep moi ham inline.
#pragma GCC optimize("-fgcse") // C: thuong da bat o O2, khong can them.
#pragma GCC optimize("-ffast-math") // C: co the pha gia thiet ve so thuc/NaN/Inf.
#pragma GCC optimize("-fsched-spec") // C: thuong khong can chinh scheduling thu cong.
#pragma GCC optimize("-funroll-loops") // B: do toc do, tang kich thuoc code.
#pragma GCC optimize("-falign-labels") // C: can chinh alignment thuong khong can.
#pragma GCC optimize("-funroll-loops") // C: trung dong unroll-loops o tren.
#pragma GCC optimize("Ofast,fast-math") // C: fast-math da nam trong Ofast; trung lap va rui ro.
#pragma GCC optimize("-fschedule-insns") // C: loi ich phu thuoc kien truc.
#pragma GCC optimize("-finline-functions") // C: compiler da tu chon inline o muc toi uu.
#pragma GCC optimize("-fstrict-aliasing") // C: thuong da bat, phai tuan thu luat aliasing.
#pragma GCC optimize("-fcse-follow-jumps") // C: toi uu cap thap, thuong khong can.
#pragma GCC optimize("-fsched-interblock") // C: scheduling phu thuoc kien truc.
#pragma GCC optimize("no-stack-protector") // C: tat bao ve stack, thuong khong dang dung.
#pragma GCC optimize("-freorder-functions") // C: thuong khong can chinh tay.
#pragma GCC optimize("-finline-small-functions") // C: thuong da duoc compiler xu ly.
#pragma GCC optimize("-finline-functions-called-once") // C: thuong da bat tu O1.
#pragma GCC optimize("-fdelete-null-pointer-checks") // C: dereference null van la UB, khong duoc dua
     vao.
#pragma GCC optimize(2) // A: O2; du neu judge da bat, khong can chep them.
```

13 Python cho lập trình thi đấu

13.1 Nhập và xuất

`input()` đọc một dòng và trả `str`. Khi thay bằng `sys.stdin.buffer.readline`, kết quả là `bytes`; `int` và `map(int, ...)` nhận được bytes, còn xử lý chuỗi Unicode thì phải `decode()`.

```
import sys
input = sys.stdin.buffer.readline

n = int(input())
a = list(map(int, input().split()))
u, v = map(int, input().split())
s = input().rstrip(b"\r\n").decode()
```

`split()` tách theo mọi khoảng trắng, không phụ thuộc số dấu cách. Nếu cần giữ khoảng trắng ở hai đầu chuỗi, chỉ bỏ ký tự xuống dòng như ví dụ; `strip()` sẽ xóa cả khoảng trắng. Khi dữ liệu chỉ gồm số và muốn đọc hết một lần:

```
it = iter(map(int, sys.stdin.buffer.read().split()))
n = next(it)
a = [next(it) for _ in range(n)]
```

Cách này giữ toàn bộ input trong bộ nhớ. Với dữ liệu lớn, đọc từng dòng. Nếu số lượng phần tử không đủ, `next(it)` phát sinh `StopIteration`.

```
print(x, y)                 # cach nhau mot dau cach
print(*a)                   # in cac phan tu tren mot dong
out = [str(x) for x in answers]
sys.stdout.write("\n".join(out))
```

Gom kết quả rồi `join` hữu ích khi có rất nhiều dòng; `join` cần các phần tử là `str`.

13.2 Vòng lặp, dãy và chuỗi

`range(r)` chạy từ 0 đến $r-1$; `range(l,r)` không chứa r. `enumerate` giúp lấy cả chỉ số và giá trị; đặt `start=1` khi đề đánh số từ 1.

```
for i in range(n):
    pass
for i in range(n - 1, -1, -1):
    pass
for i, x in enumerate(a, start=1):
    pass

a = [0] * n
grid = [[0] * m for _ in range(n)]
part = a[l:r]               # chi so l,...,r-1
rev = a[::-1]
```

Không tạo ma trận bằng `[[0] * m] * n`: các hàng khi đó trỏ tới cùng một list. `a[l:r]` tạo list mới.

```
words = "a b c".split()
s = " ".join(words)
letters = list(s)
code = ord('A')             # 65
ch = chr(65)                # 'A'
```

13.3 Sắp xếp và tìm vị trí

`sort()` sửa list tại chỗ và trả `None`; `sorted()` trả list mới. Python sắp xếp ổn định: những phần tử có khóa bằng nhau giữ nguyên thứ tự cũ.

```
a.sort()
a.sort(reverse=True)
pairs.sort(key=lambda p: (p[1], -p[0]))
b = sorted(a)

from bisect import bisect_left, bisect_right
i = bisect_left(a, x)       # dau tien >= x
j = bisect_right(a, x)      # dau tien > x
count_x = j - i
```

Hai hàm `bisect` yêu cầu đoạn đang xét đã sắp xếp theo cùng quy tắc; trả vị trí chèn trong $[0, n]$.

13.4 Từ điển, tập hợp, hàng đợi và heap

```
from collections import Counter, defaultdict, deque
from heapq import heappush, heappop

freq = Counter(a)
cnt = freq[x]                # 0 neu x chua xuat hien
g = defaultdict(list)
g[u].append(v)
seen = set()
seen.add(u)
q = deque([u])
q.append(v)
first = q.popleft()
heap = []                    # min-heap
heappush(heap, (dist, v))
dist, v = heappop(heap)
```

Với `dict` thường, dùng `d.get(key, 0)` để nhận giá trị mặc định mà không tạo khóa mới. `defaultdict(list)` tạo list rỗng khi truy cập khóa chưa có. Heap Python là min-heap; muốn lấy số lớn nhất có thể đẩy `-x` rồi đổi dấu khi lấy ra.

13.5 Các lỗi cú pháp dễ gặp

- Python dùng thụt lề để xác định khối lệnh; điều kiện và vòng lặp kết thúc phần đầu bằng dấu `:`.
- `/` là chia số thực; `//` là chia lấy sàn, khác phép cắt về 0 khi số âm. `**` là lũy thừa, `^` là XOR, không phải lũy thừa.
- `map(...)` và `zip(...)` trả iterator; chuyển thành `list(...)` nếu cần truy cập nhiều lần.
- Kiểm tra giới hạn thời gian trước khi dùng Python cho vòng lặp rất lớn hoặc thuật toán đệ quy sâu. Số nguyên lớn, `pow` modulo và các hàm `math` xem mục 12.5.

Once a Djm, always a Djm. Since 2024